

以KV 缓存为中心的高效 长文本方法的优化和实践

演讲人：姜慧强

Microsoft Research Asia / Research SDE

AiCon
全球人工智能开发与应用大会

■ About me

- Research SDE in Microsoft Research Asia (Shanghai)
- System-Algorithm Co-design
- Efficient methods to accelerate inference/training



目录

- 01 长文本大语言模型的应用和推理挑战
- 02 当前主流推理优化方法与技术
- 03 以KV缓存为中心的大语言模型推理架构
- 04 以KV缓存为中心的高效长文本方法
- 05 总结与展望

📍 北京

QCon

全球软件开发大会

会议时间：4月10-12日

- 大模型赋能 AIOps
- 越挫越勇的大前端
- 云上业务架构演进
- 多模态大模型及应用
- 海外 AI 应用创新实践

📍 北京

AiCon

全球人工智能开发与应用大会

会议时间：6月27-28日

- 端侧智能
- AI Agent
- 多模态大模型
- 金融+大模型

📍 上海

QCon

全球软件开发大会

会议时间：10月23-25日

- AI Agent
- 大模型训练推理
- 端侧 AI
- 搜推深度融合
- 数智金融

4月

5月

6月

8月

10月

12月

📍 上海

AiCon

全球人工智能开发与应用大会

会议时间：5月23-24日

- 通用大模型
- AI Agent
- 垂直领域应用
- 模型可解释性

📍 深圳

AiCon

全球人工智能开发与应用大会

会议时间：8月22-23日

- 模型效率与部署
- 多模态大模型
- 大模型安全
- 智能硬件

📍 北京

AiCon

全球人工智能开发与应用大会

会议时间：12月19-20日

- 通用大模型
- 智能硬件
- LMOPs
- 具身智能

极客邦科技 2025 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询



查看会议

01 长文本大语言模型的应用和推理挑战

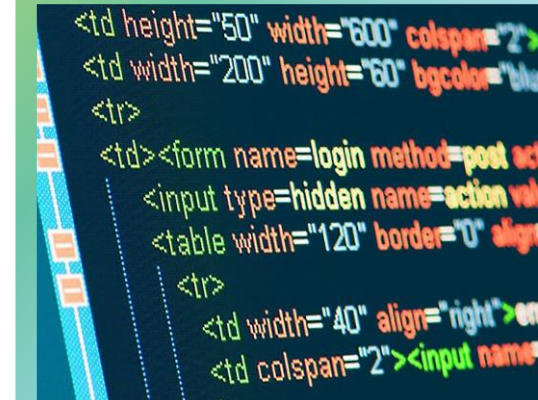
Longer Context, Greater Intelligence



- Massive Pages of Docs



- Extended Meeting Time



- Lengthy Codebases



Complex Reasoning



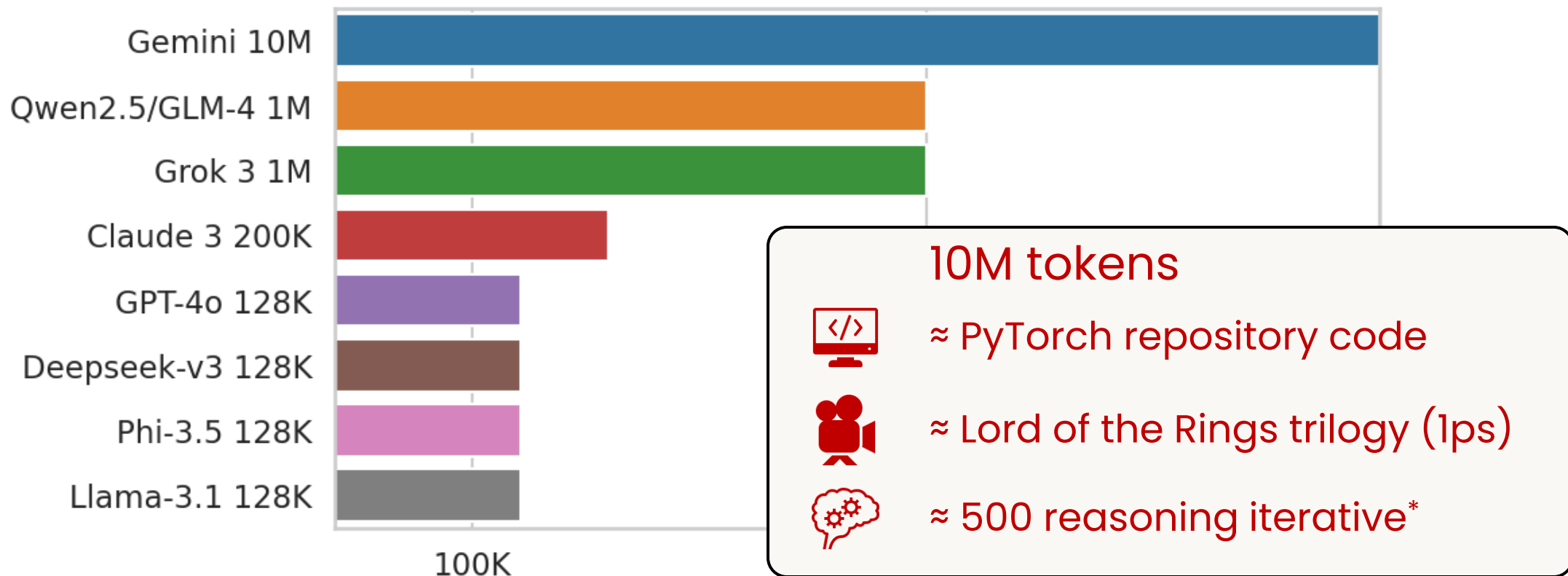
Endless Agentic History



Lifelong Personalization

Longer Context: A Cornerstone of AGI

- Almost all latest models can process contexts exceeding 100K tokens.

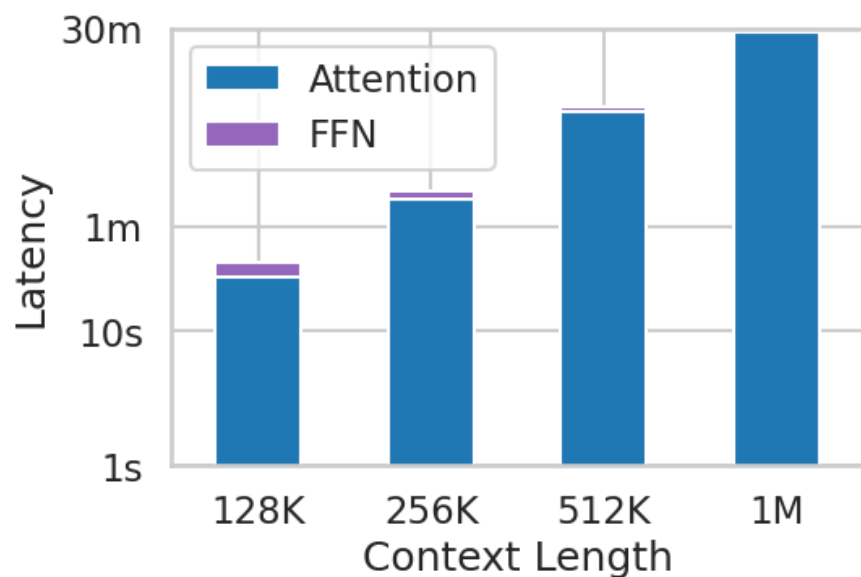


<https://lifearchitected.ai/models/#context-windows>

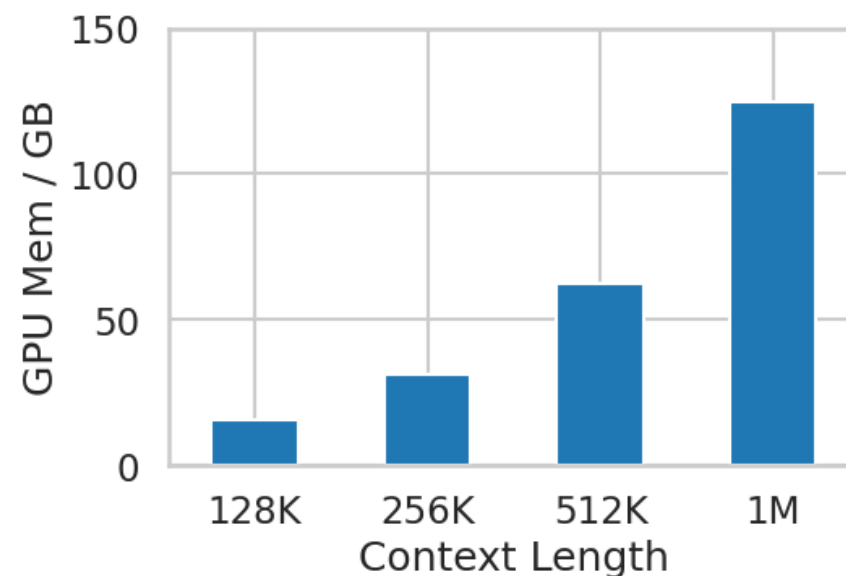
[DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning](#)

Long-context LLMs Inference is Expensive and Slow

- ❑ **Long Prefilling Latency**, 30 minutes to process 1M tokens on an A100 for an 8B LLM.
- ❑ **Large GPU Memory Consumption**, 62GB of GPU memory is required for 512 K tokens in fp16.

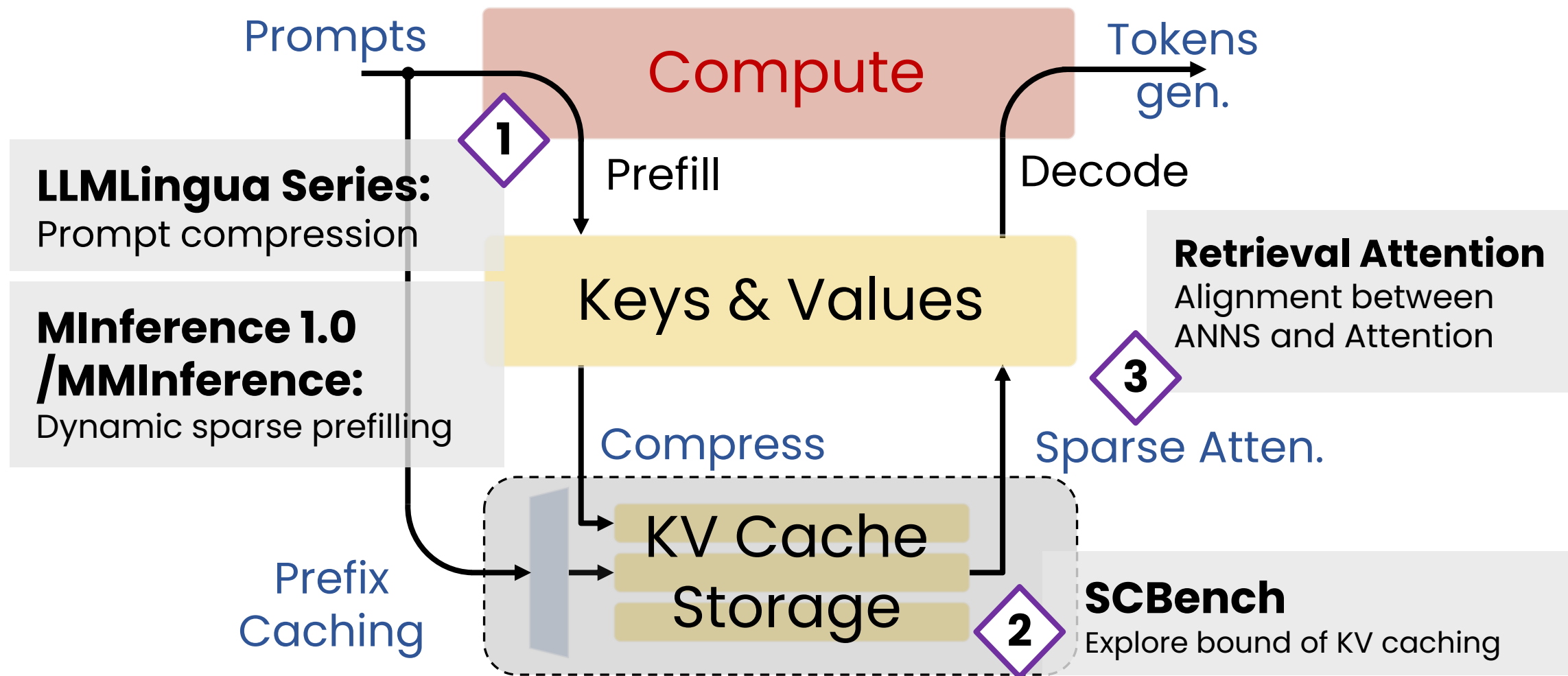


Long Prefilling Latency
=> **MI**nference



Large GPU Memory Consumption
=> **RetrievalA**ttention

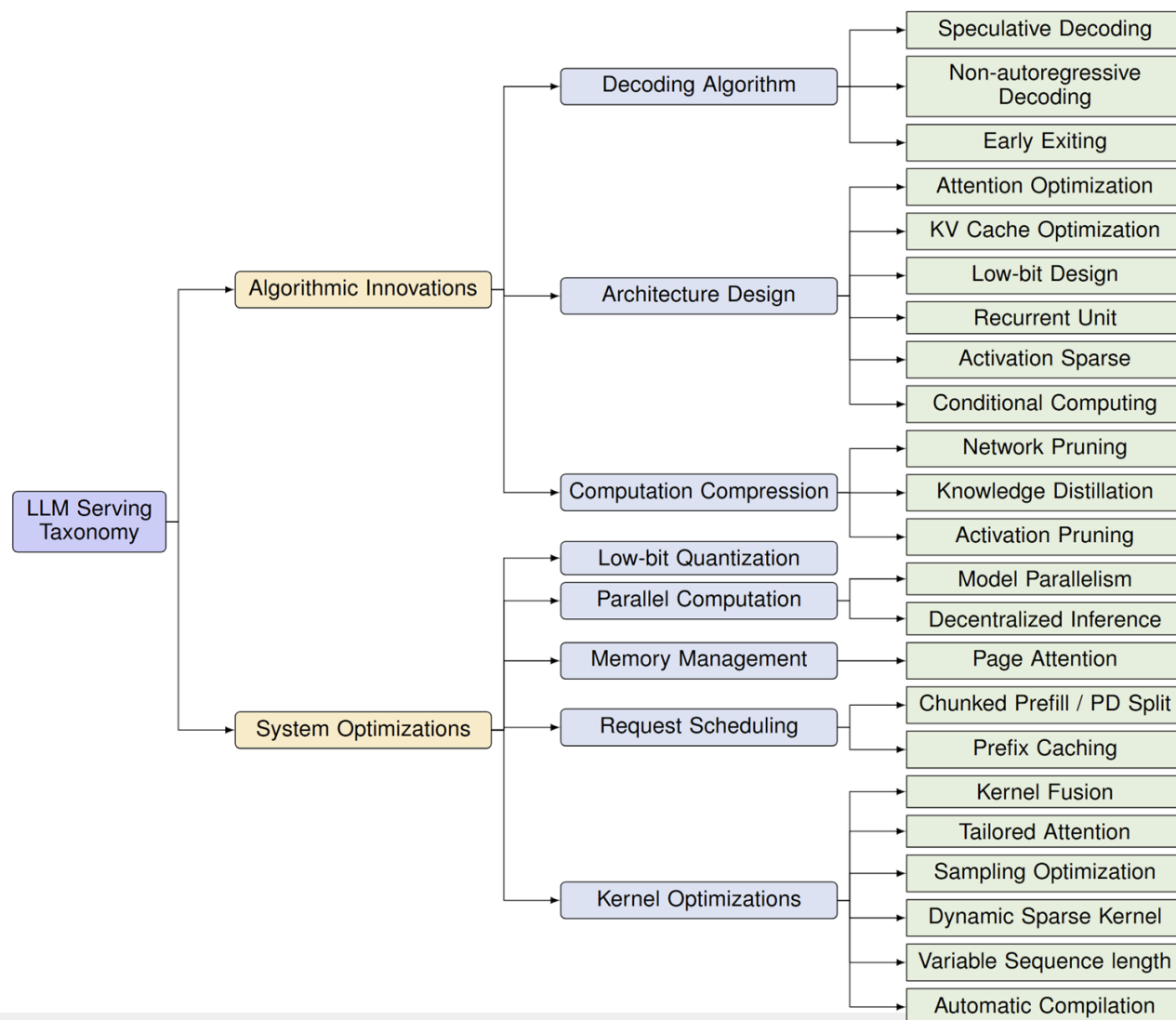
Opportunities to Improve



02

当前主流推理优化方法与技术

LLMs Inference优化方法



Prefix Caching



RadixAttention



Automatic Prefix
Caching



Prompt Caching



Context Caching



Prompt Caching

(a) Prefix caching is widely used in LLM framework.

(b) Prefix caching is widely used in LLM API.

03 以KV缓存为中心的大语言模型 推理架构

以KV缓存为中心的大语言模型推理架构

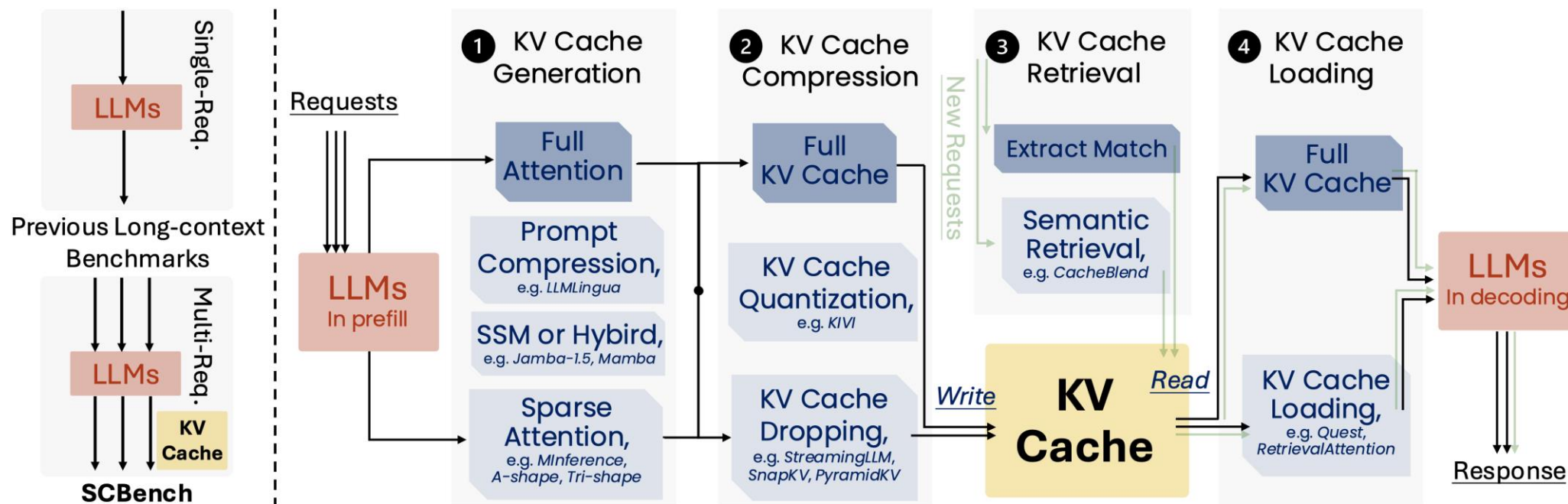


Figure 1: KV Cache lifecycle. Prior benchmarks focus on single-request, while real-world applications reuse KV cache across requests. We propose **SCBench** and categorize long-context methods into KV Cache Generation, Compression, Retrieval, and Loading from a KV-cache-centric perspective.

KV Cache is not limited to a single turn.

- ❑ Long-context methods are designed and utilized around the KV cache, but existing benchmarks focus only on single-request scenarios, ignoring its full lifecycle in real-world use.



Repo-level Code
Debugging/ Long-
document QA



RadixAttention



Prompt Caching



Multi-turn Dialogue



Context Caching



Self-play Reasoning



Automatic Prefix
Caching



Prompt Caching

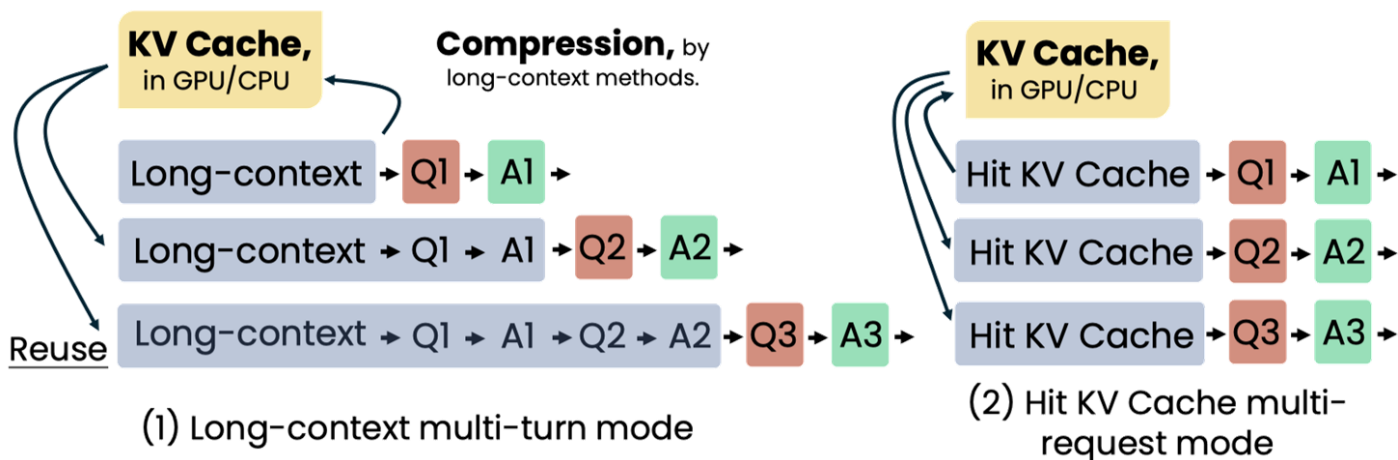
(a) Long-Context is shared in
real-world scenarios.

(b) Prefix caching is widely used in
LLM framework.

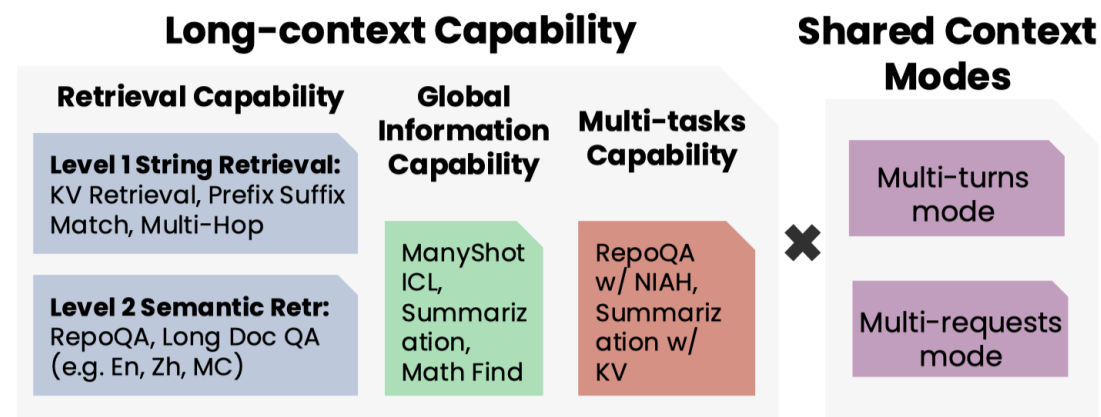
(c) Prefix caching is widely used
in LLM API.

SCBench

- ❑ Two typical shared context modes;
- ❑ Four category long-context capability;



(a) Two Shared Context Modes



(b) Overview of SCBench

■ SCBench–benchmark building

□ 12 subtasks;

Task	Description	Capability	Avg. Input Length	Avg. Output Length	#Sessions / #Turns
Retr.KV	Key-value retrieval from many key-value pairs	String Retrieval	125K	943	100/500
Retr.Prefix-Suffix	Find string with specific prefix and suffix in a dict	String Retrieval	112K	914	100/500
Retr.MultiHop	Tracking variables assignment in a long input	String Retrieval	124K	410	90/450
Code.RepoQA	Functions retrieval from a GitHub repo	Semantic Retrieval	65K	6,058	88/440
En.QA	English Question Answering	Semantic Retrieval	198K	272	69/351
Zh.QA	Chinese Question Answering	Semantic Retrieval	1.5M	322	35/189
En.MultiChoice	English Multi-Choice Questions	Semantic Retrieval	188K	215	58/299
Math.Find	Math computation tasks within long sequence arrays	Global Information	120K	172	100/240
ICL.ManyShot	Hundreds-shot in-context learning	Global Information	22K	975	54/270
En.Sum	Summarize a doc given multiple docs as input	Global Information	104K	1,170	79/350
Mix.Sum+NIAH	Multi-tasking of En.Sum and Needle in A Haystack	Multi-tasking	105K	3,441	70/560
Mix.RepoQA+KV	Multi-tasking of RepoQA and KV retrieval	Multi-tasking	68K	5,318	88/704
Total	-	-	227K	1,684	931/4,853

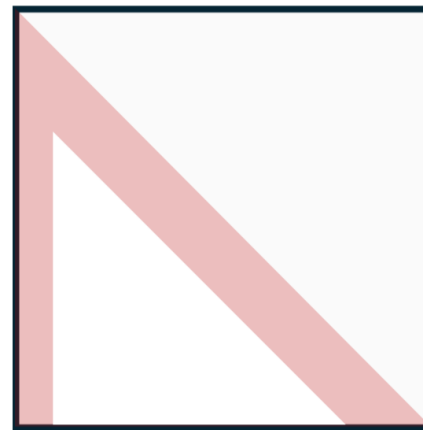
SCBench–Long–context Methods

□ 13 long-context methods;

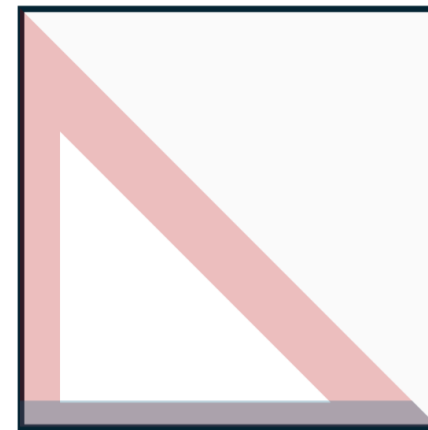
Methods	Taxonomy	Stage	P-stage Efficient	D-stage Efficient	KV Cache Size	Prefilling Complexity	Decoding Complexity
Codestral Mamba (team, 2024)	Gated Linear RNN	❶	✓	✓	$O(k)$	$O(kn)$	$O(km)$
Jamba (Lieber et al., 2024)	Gated Linear RNN + Full Attention	❶	✓	✓	$O(n)$	$O(n^2)$	$O(nm)$
LLMLingua-2 (Pan et al., 2024)	Prompt Compression	❶	✓	✗	$O(\alpha n)$	$O(\alpha^2 n^2)$	$O(\alpha nm)$
A-shape (Xiao et al., 2024b) Tri-shape MInference (Jiang et al., 2024)	Sparse Attention	❶	✓	✗	$O(n)$	$O(kn)$	$O(nm)$
StreamingLLM (Xiao et al., 2024b) SnapKV (Li et al., 2024c) PyramidKV (Cai et al., 2024)	KV Cache Dropping	❷	✗	✓	$O(k)$	$O(n^2)$	$O(km)$
KIVI (Liu et al., 2024e)	KV Cache Quantitation	❷	✗	✓	$O(n)$	$O(n^2)$	$O(nm)$
CacheBlend (Yao et al., 2024a)	KV Cache Retrieval	❸	✓	✗	$O(n)$	$O(n^2)$	$O(nm)$
Quest (Tang et al., 2024) RetrievalAttention (Liu et al., 2024b)	KV Cache Loading	❹	✗	✓	$O(n)$	$O(n^2)$	$O(km)$

Tri-shape

□ last window query region + A-shape;



① A-shape head



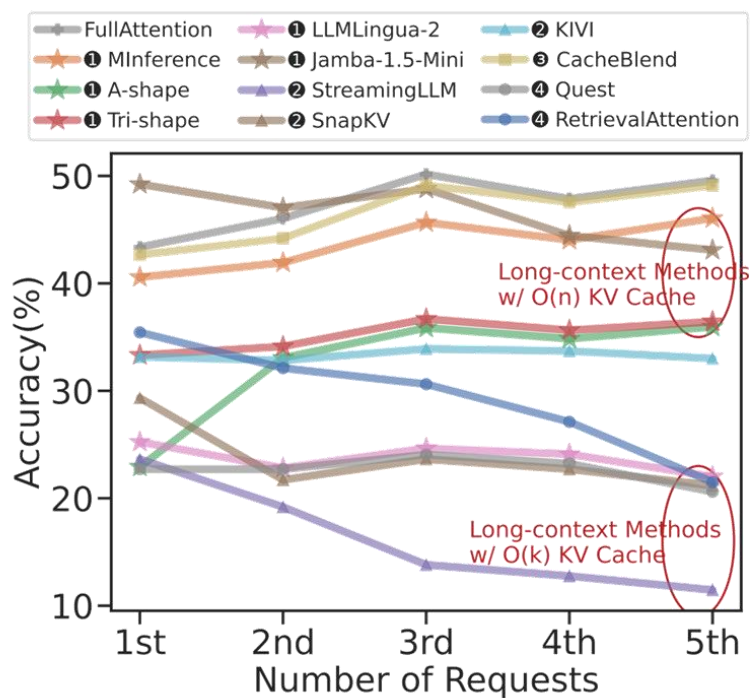
② Tri-shape head

Table 17: Case Study of Retr.KV to compare A-shape and Tri-shape.

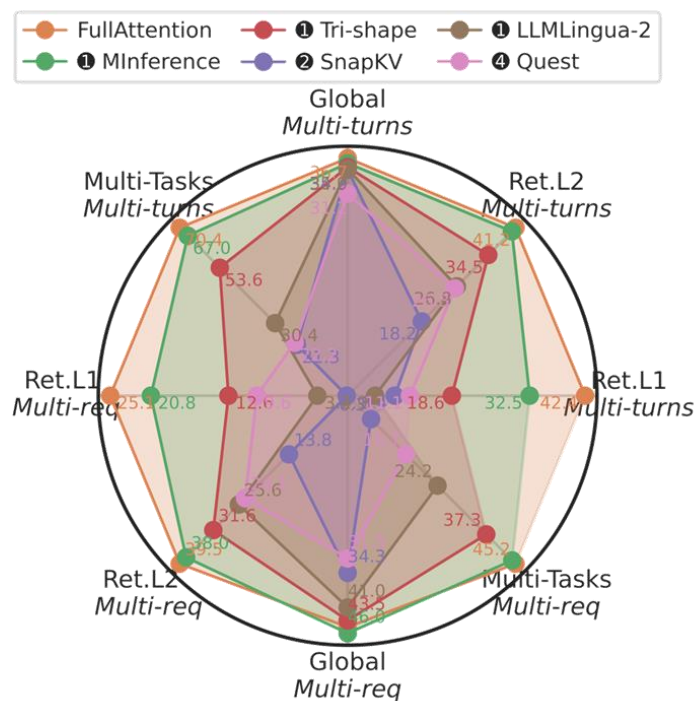
Model	Response
Ground Truth	Turn 1: cb59052b-9128-4979-9c0e-e1de4adcf73b Turn 2: a9ee6345-2331-4914-b32b-25a780931d45
Llama-3.1-70B	Turn 1: 798c2306-5ad1-42a3-a8d7-3f5867f260c9: 5e369d13-ce48-40f9-824d-a55ef5748e9e, 5e257563-f131-4583-b72e-bd43b07a4e09: d768cb61-186a-4a56-bbbc-1c94f2d2597e, ff8fd84d-b1f8-4e94-b3a3-cf5f252541f9: 45e9039a-01ec-4fd3-a22e-098d1c3a2 Turn 2: e69af07e-d786-43ea-8cc4-74c452b3d66793c5c 4a0d0b3 d2e72f0c2e0b7b2a4e7f5e0d0 b7b2a4e7f5e0d0 b7b2a4e7f5e0d0b7b2a4 e7f5e0d0b7b2 a4e7f5e0d0b7b2a4e 7f5e0d0b7b2a4e7f5e0d0b7b2a4e7f
Llama-3.1-70B w/ A-Shape	Turn 1: assistant<lend_header_id>assistant<lend_header_id> assistant<lend_header_id> assistant<lend_header_id>assistant <lend_header_id>assistant<lend_header_id> assistant<lend_header_id>assistant<lend_header_id>assistant Turn 2: 3ebf05c3-35af-483b-b46e-1bf3c67f3682 : a4a5d7a3-8a3c-4b3a-8c6d-7c0d1a4a4a
Llama-3.1-70B w/ Tri-Shape	Turn 1: 6a6e0b3e-3d7c-4f33-ba46-7f42bb75b03f: 1f5eba0d-5ccf-4262-aa76-d7fbabdc0b9a Turn 2: 3ebf05c3-35af-483b-b46e-1bf3c67f3682: 1f5eba0d-5ccf-4262-aa76-d7fbabdc0b9a

Performance in Multi-turn and Capability

- ❑ Sub- $O(n)$ Memory is Almost Infeasible in Multi-Turn Decoding.
- ❑ Long-generation scenarios exhibit distribution shift issues.

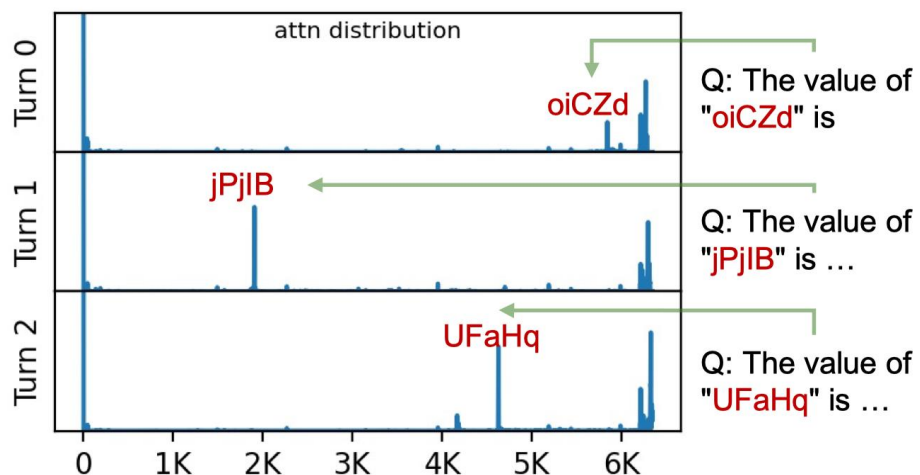


(a) Performance Across Different Requests

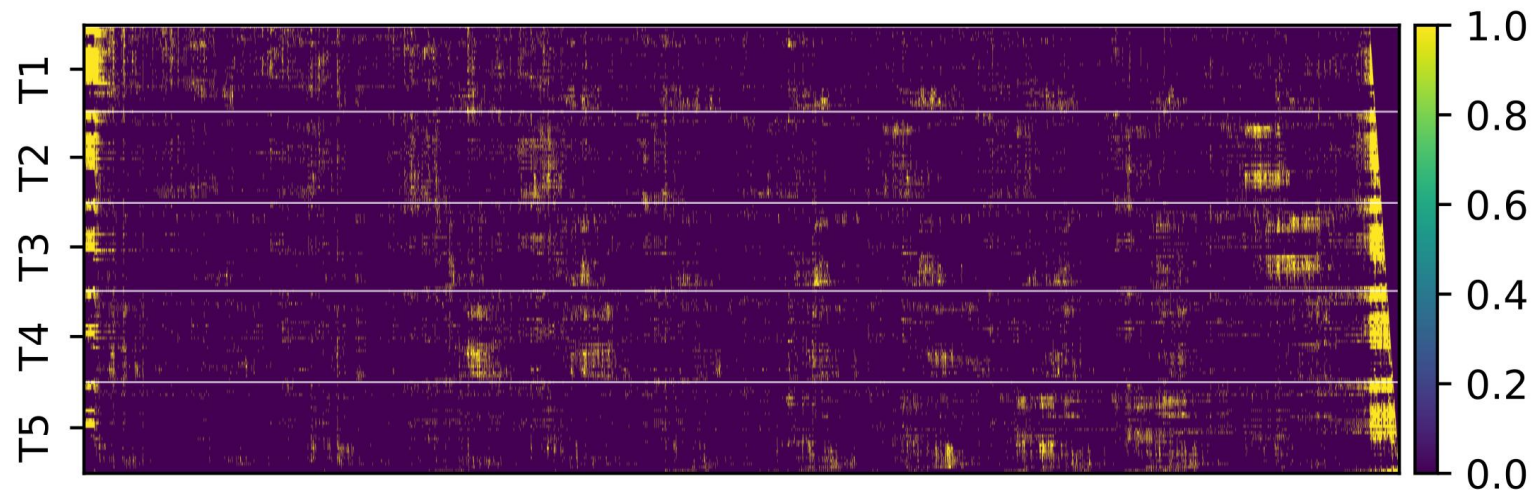


(b) Performance in Different Abilities

Long-generation scenarios exhibit distribution shift issues.



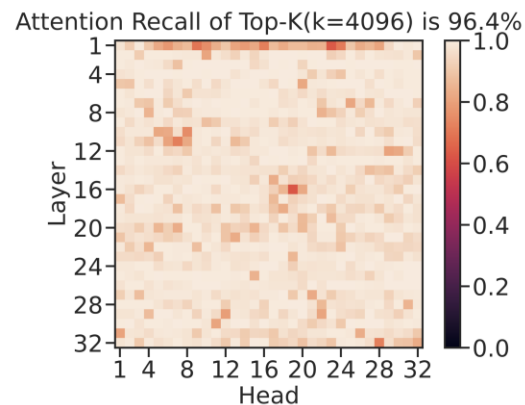
(a) Critical KVs Vary Across Queries



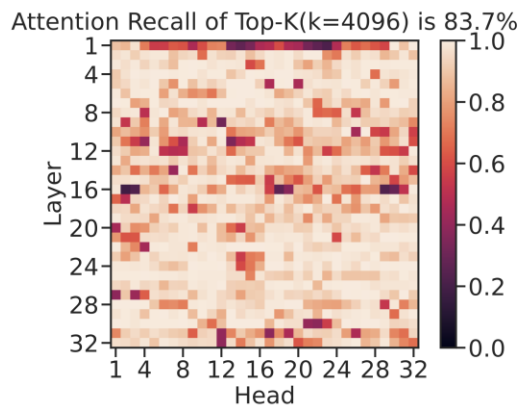
(b) Attention Map of Retr.KV Across Turns

04 以KV缓存为中心的高效长文本 方法

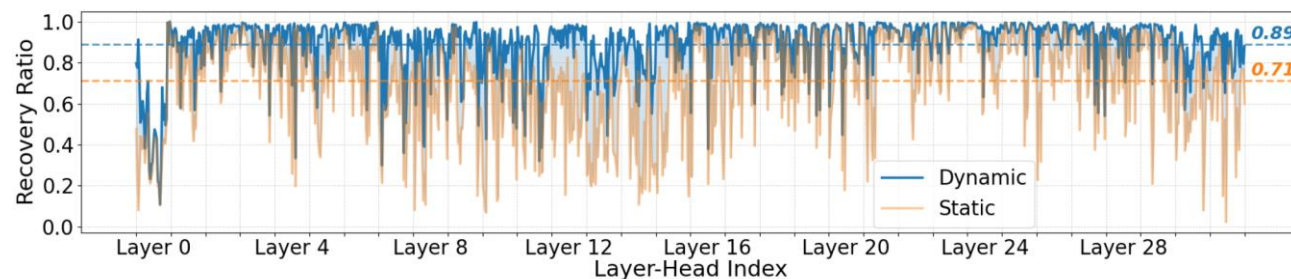
Observation 1: Attention is Dynamically Sparse



(a) Attention is sparse.



(b) Sparsity of attention is dynamic.



(c) Dynamically sparsity in decoding.

in prefilling

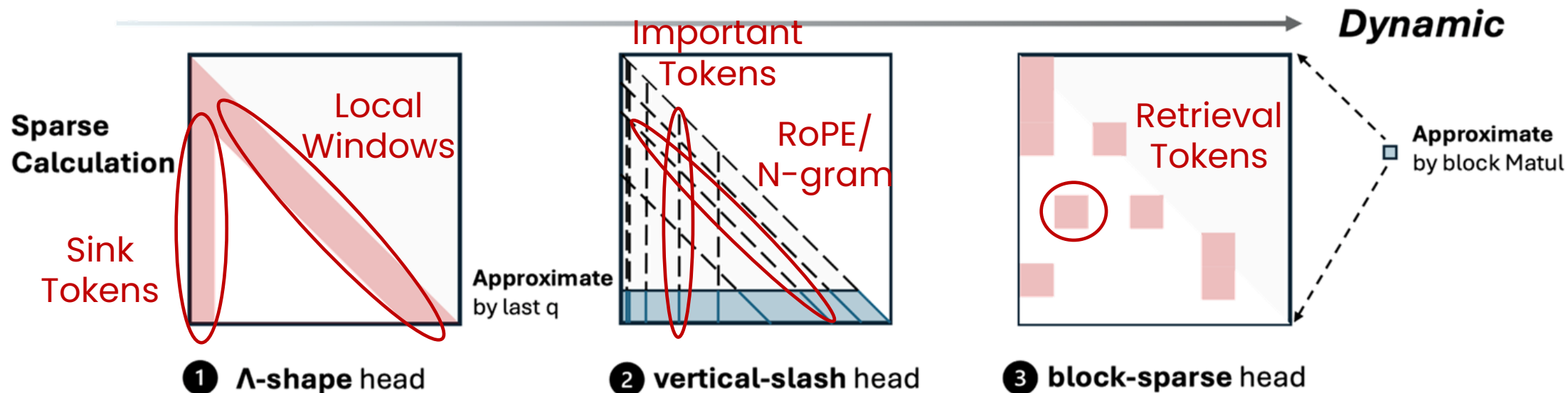
- **Figure. Dynamic sparsity in attention:** (a) Top-k (k=4096) columns cover a significant portion of attention scores in a 128k context. (b) Fewer scores are retrieved when reusing top-k indices across examples, highlighting its dynamic nature. Visualizations use LLaMA-3-8B on a single A100.

in decoding

Figure. (c) Dynamic sparsity in LLaMA-3-8B decoding in KV retrieval (100k tokens): Dynamically selecting top-1000 tokens achieves 89% recovery, while static selection drops to 71%.

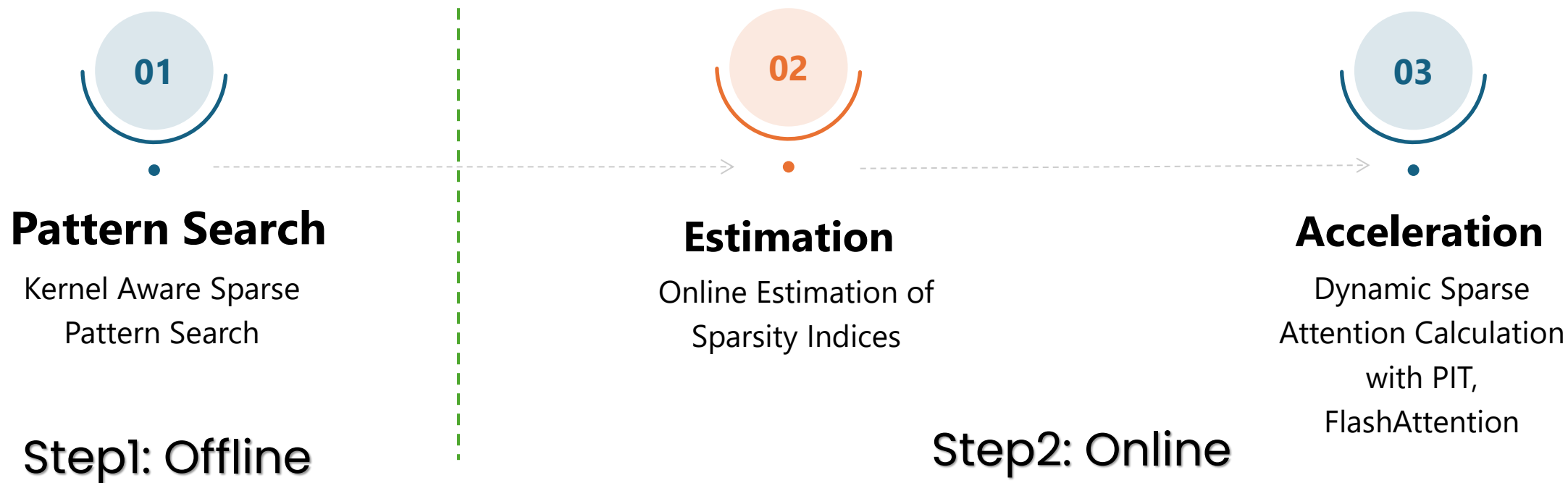
■ Observation 2: Attention Sparsity Exhibits Patterns

- ❑ After pretraining, attention **exhibits various sparse patterns**, including A-shape, Vertical-Slash, and block-sparse patterns.
- ❑ These sparse **patterns are fixed** for each head across different inputs.
- ❑ The specific **sparse elements** (e.g., column index, slash index) **dynamically change** depending on the context.

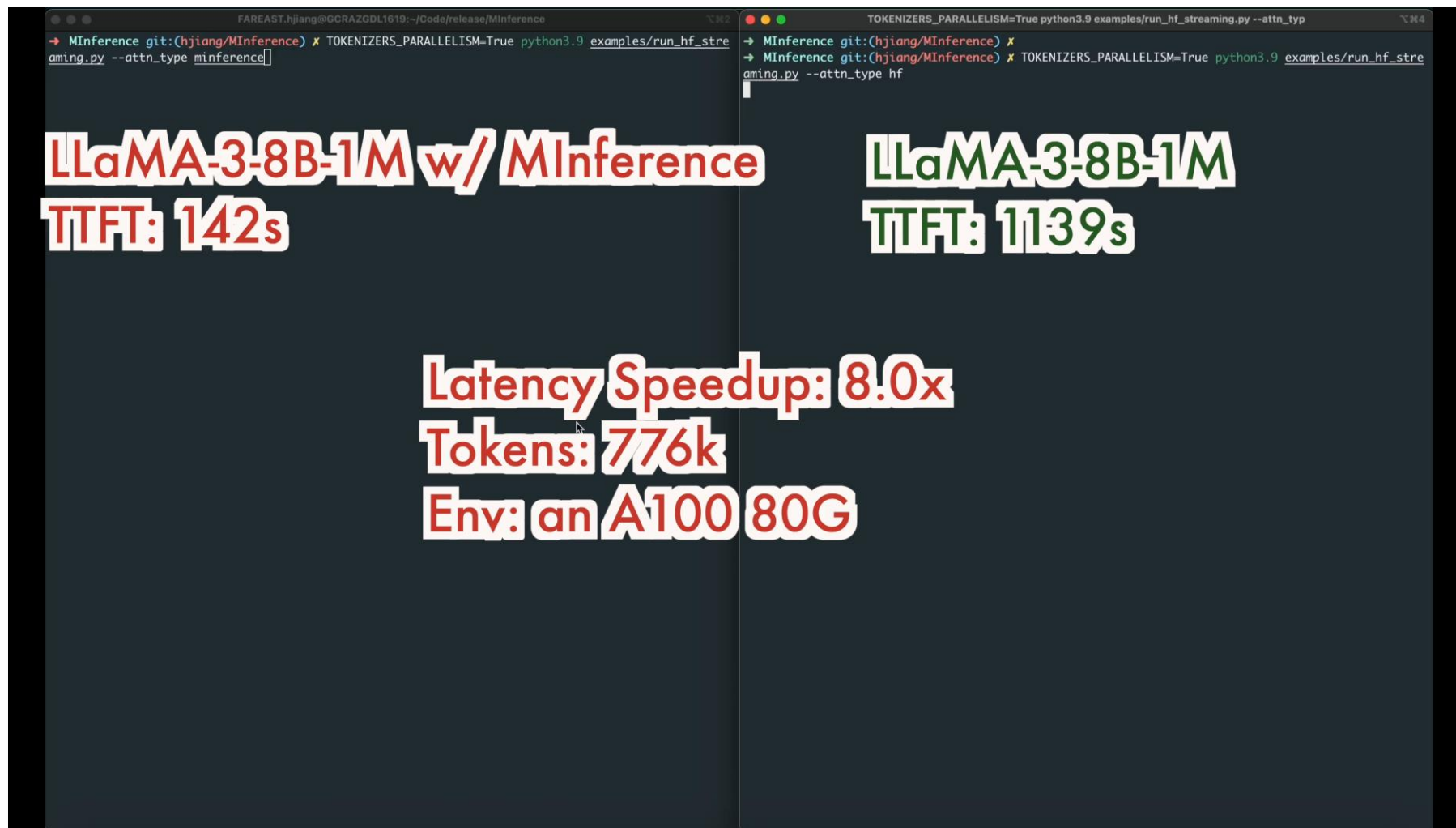


MInference 1.0

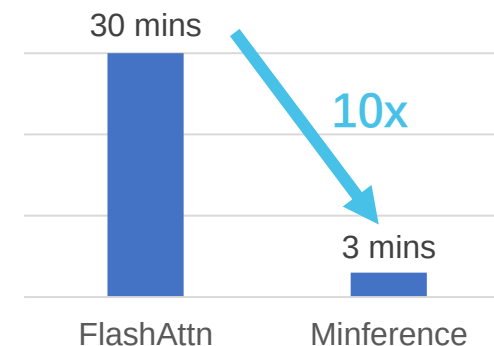
- MInference utilizes the inherent dynamic sparsity found in LLMs alongside optimized GPU kernel design to enhance the TTFT.



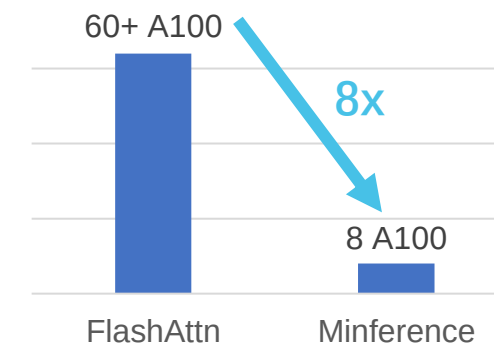
MInference 1.0



Latency at 1M tokens on a single A100

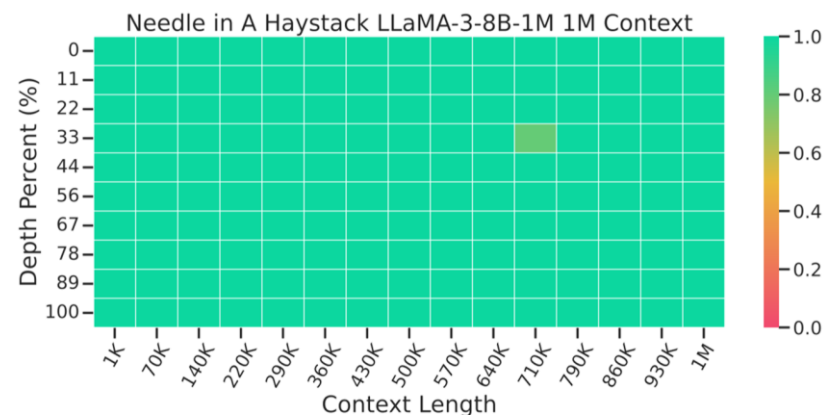


GPUs needed for sub-20s latency at 1M tokens

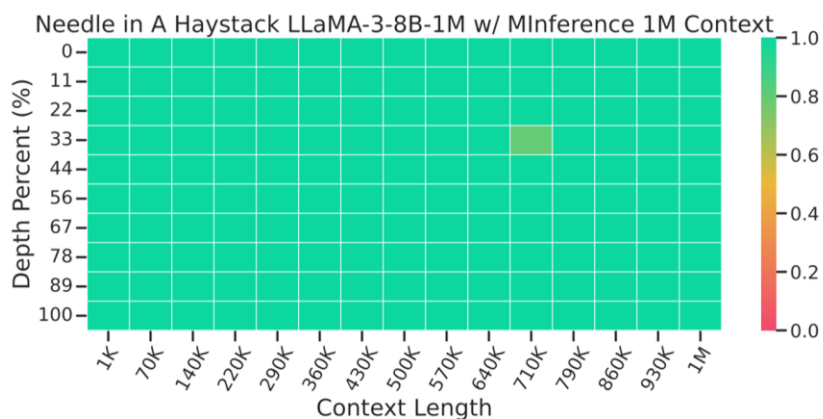


How effective and efficient is MInference?

• 1. NIAH

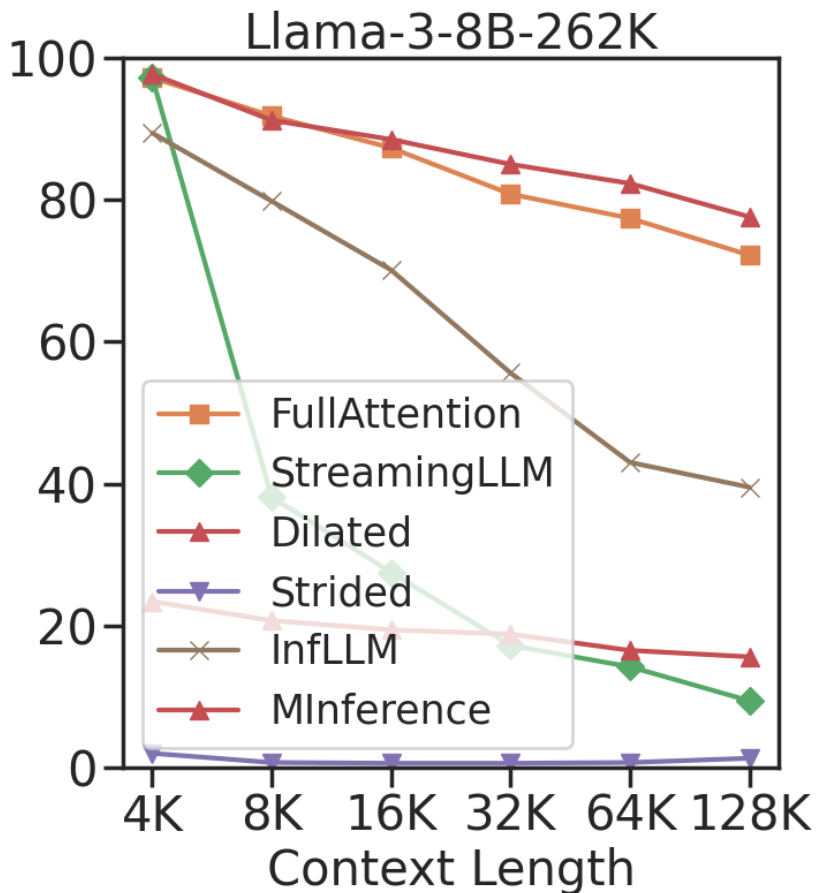


(a) LLaMA-3-8B

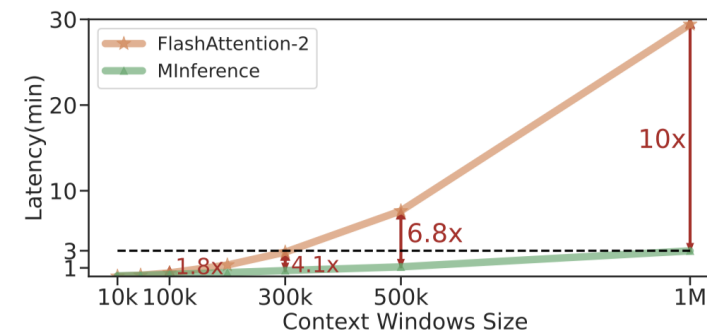


(b) LLaMA-3-8B w/ MInference

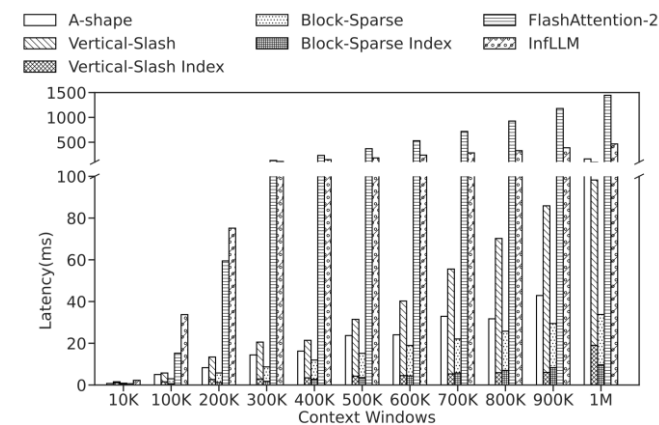
2. RULER Avg Tokens: 4K-128K



3. Latency Benchmark

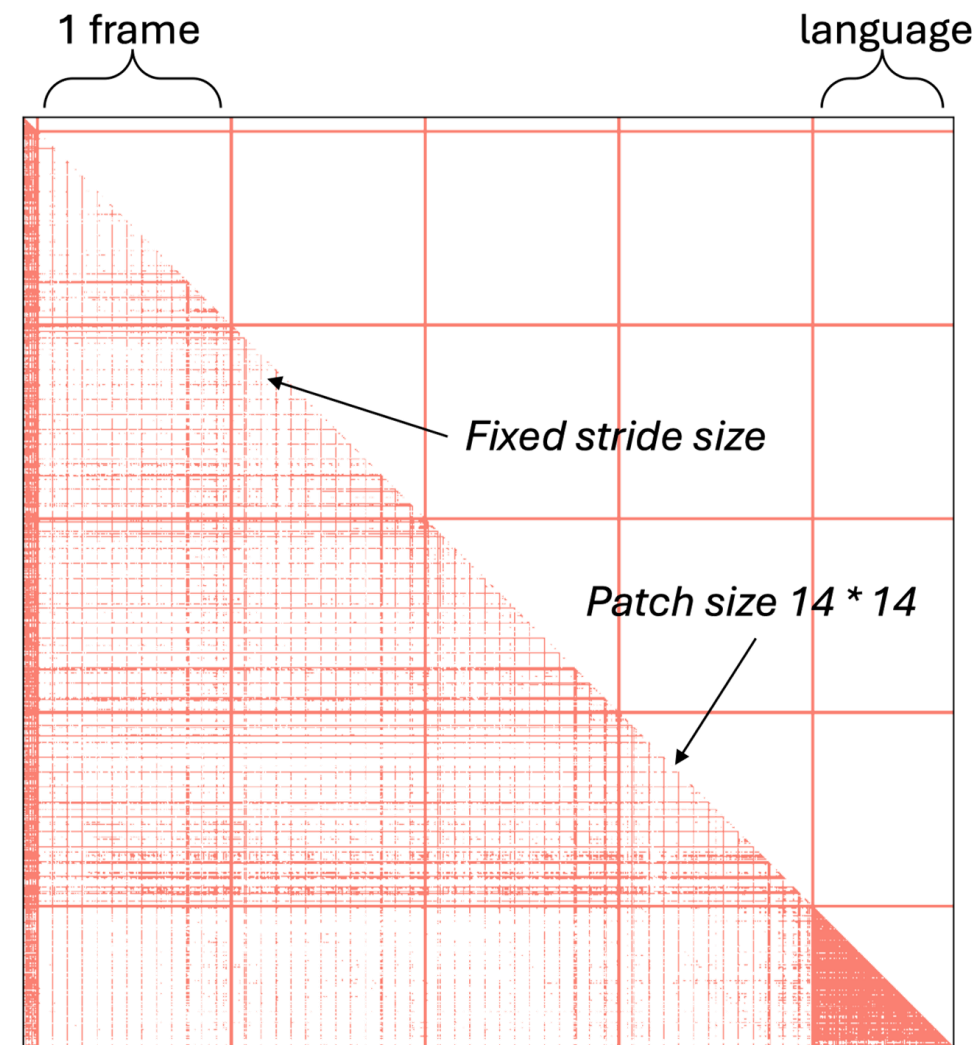


(b) Latency Speedup



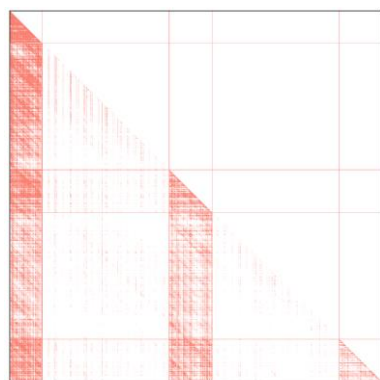
■ Observation 3: Grid Head in VLMs

- ❑ Local tokens in **temporal** and **spatial** dimensions are evenly distributed within the attention map.
- ❑ Stride and starting position vary with context, the horizontal and vertical lines are evenly spaced and often symmetrical.

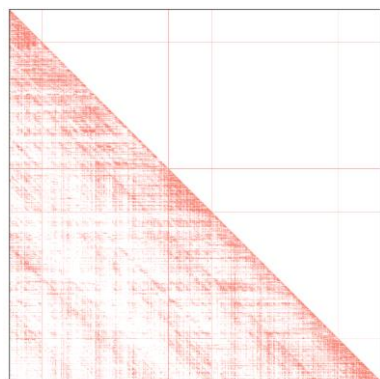


■ Observation 4: Modality Boundaries in Multi-Modal Input

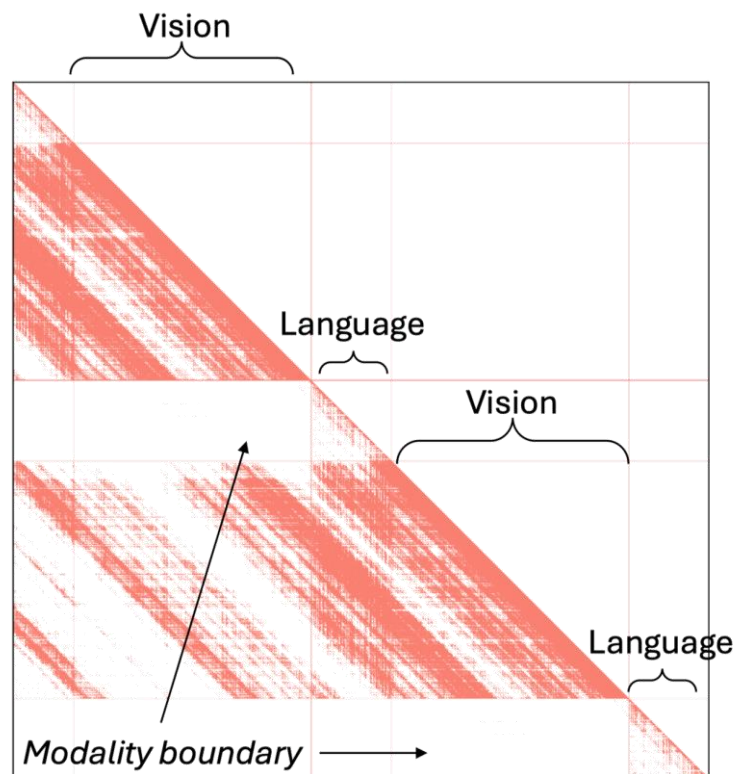
- ❑ 1) Intramodality consistency; 2) Modality-separated continuity



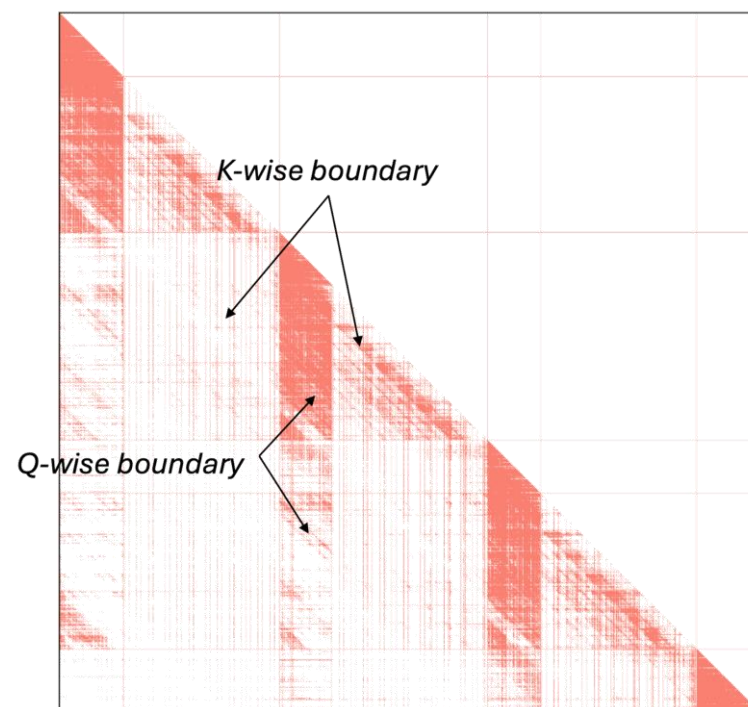
(a) K-Boundary pattern.



(b) No-Boundary pattern.



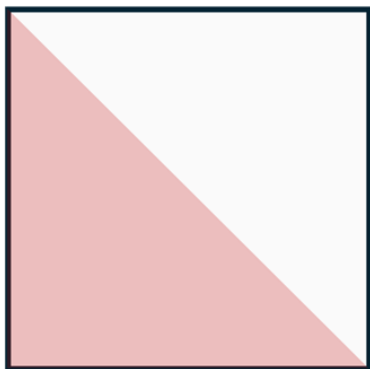
(b) Q-Boundary pattern.



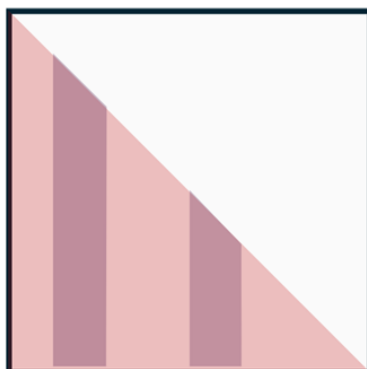
(c) 2D-Boundary pattern.

MMInference

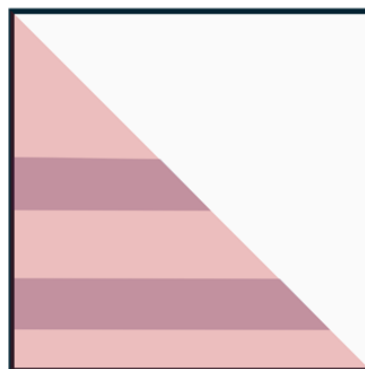
Inter-modality Attention Pattern



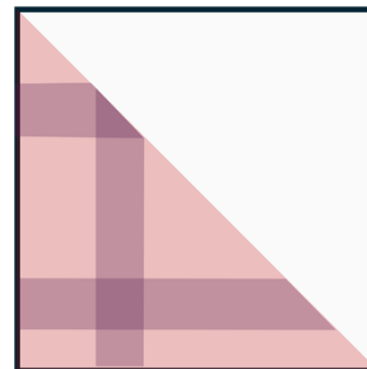
① No-Boundary head



K-Boundary head

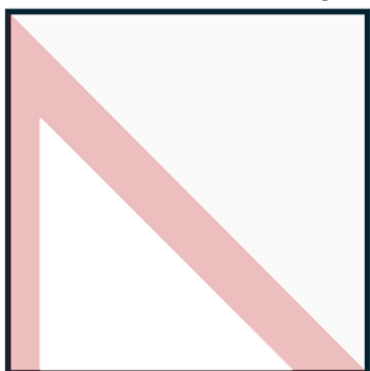


② Q-Boundary head

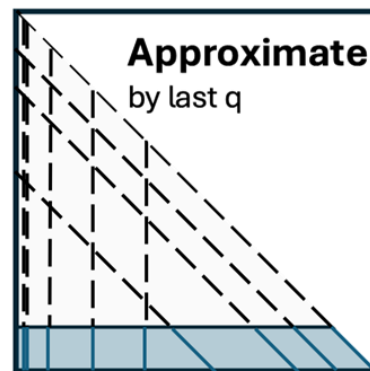


③ 2D-Boundary head

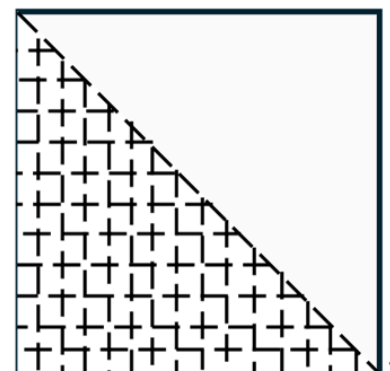
Intra-modality Attention Pattern



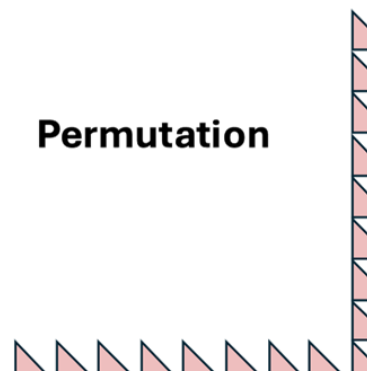
① Lambda-shape head



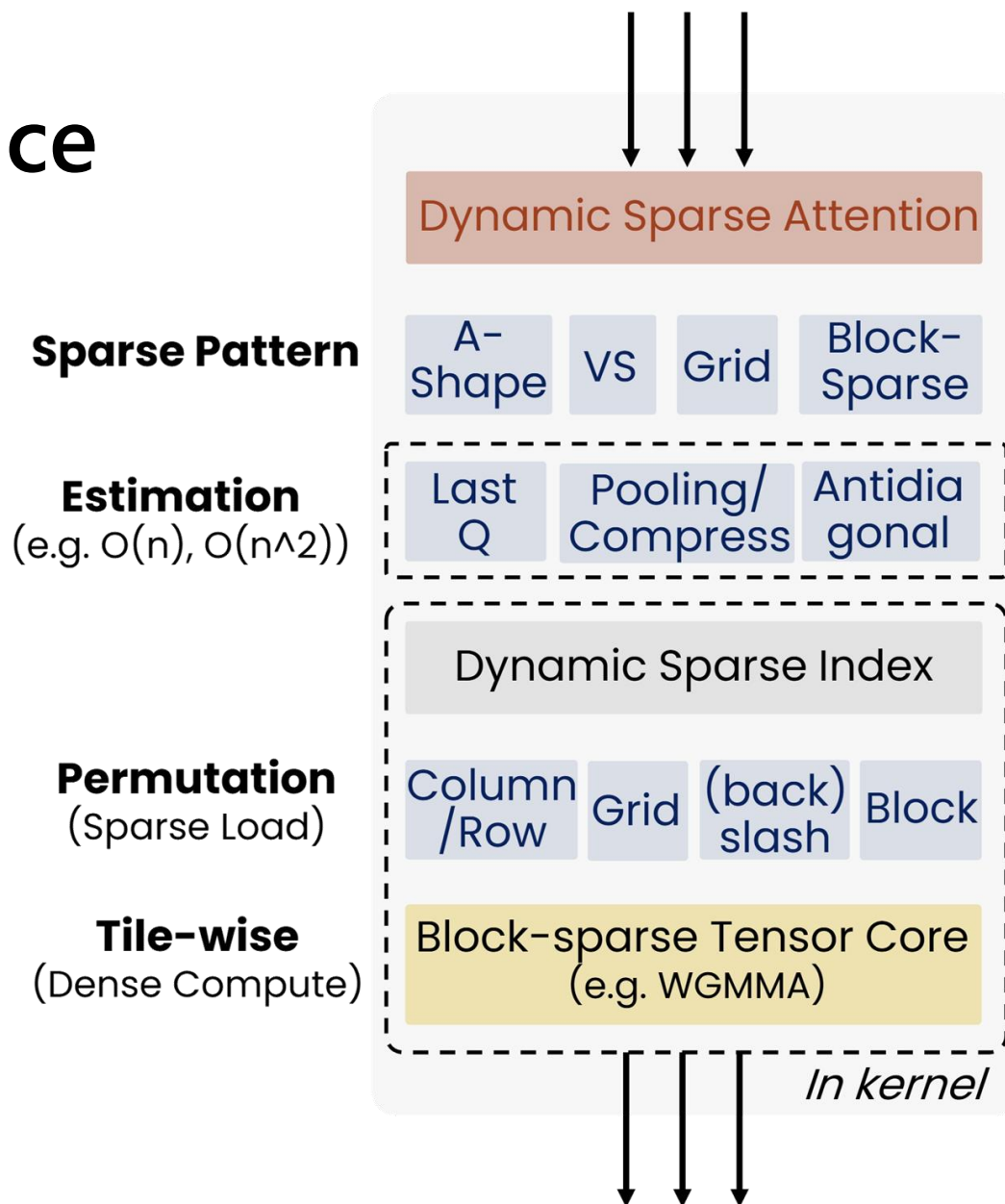
② vertical-slash head



③ grid head



MMInference



MMInference: Grid Head in Multi-Modality

Algorithm 1 Grid Head

Input: $Q, K, V \in \mathbb{R}^{S \times d_h}$, stride space $s_g \in \phi_g$

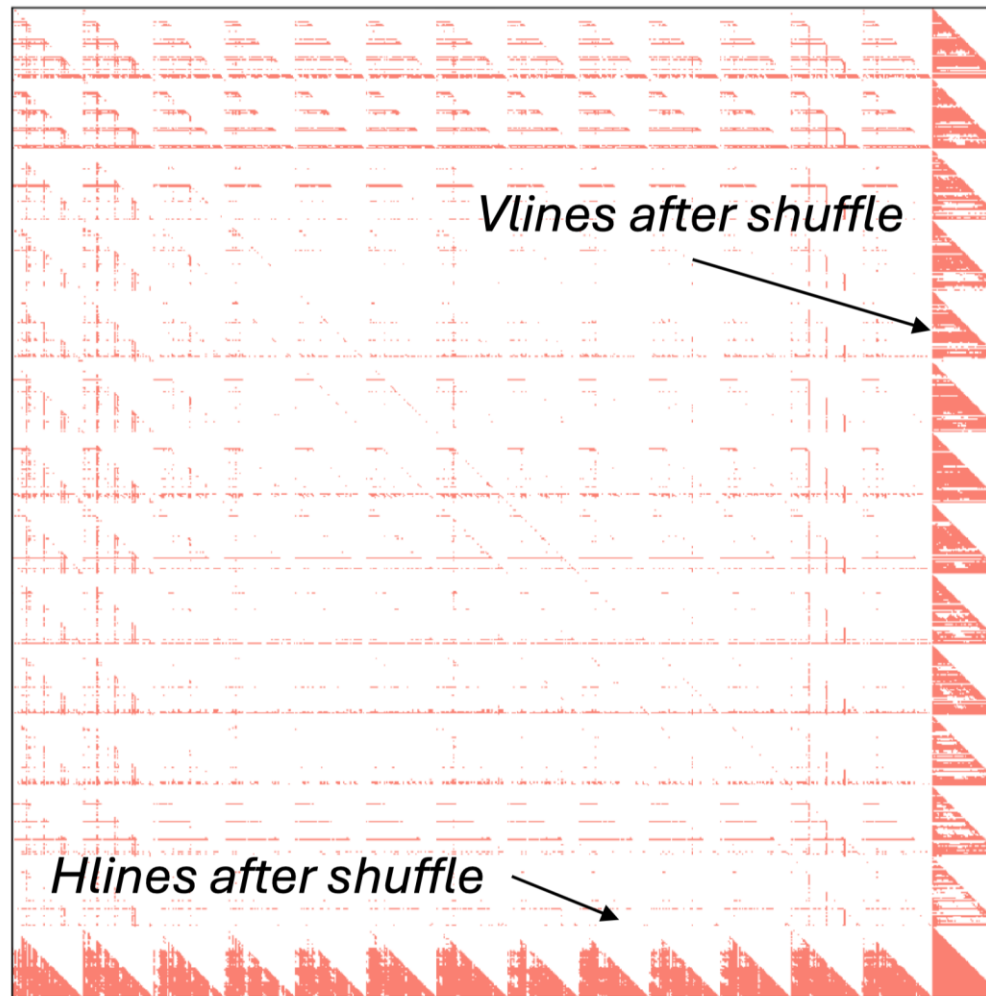
Approximate stride and phase (last_q = 64)
 $\hat{A} \leftarrow \text{softmax} \left(Q_{[-\text{last_q:}]} K^\top / \sqrt{d} + m_{\text{casual}} \right)$

Online search grid stride and phase
 $b_r \leftarrow 0$
for $i \leftarrow 1$ to $|\phi_g|$ **do**
 if $\max(\text{view}(\hat{A}, s_{g,i})) > b_r$ **then**
 $s_g \leftarrow s_{g,i}, p_g \leftarrow \text{argmax}(\text{view}(\hat{A}, s_{g,i}))$
 $b_r \leftarrow \max(\text{view}(\hat{A}, s_{g,i}))$
 end
end for

Permute Q, K, V tensors
 $\overline{Q}, \overline{K}, \overline{V} \leftarrow \text{permute}(Q), \text{permute}(K), \text{permute}(V)$

Final dynamic sparse attention scores w/ FlashAttention (only the last and rightmost block)
 $A \leftarrow \text{softmax} \left(\text{sparse}(\overline{Q} \overline{K}^\top, s_g, p_g) / \sqrt{d} \right)$

Sparse mixed scores and values
 $y \leftarrow \text{sparse}(A \overline{V}, s_g, p_g)$
return y



MMInference: Q-Boundary pattern

Algorithm 2 Q-Boundary Head

Input: $Q, K, V \in \mathbb{R}^{S \times d_h}$, modality type index i_m , modality type set $m \in \phi_m$

Permute Q tensors based on modality

$\bar{Q} \leftarrow \text{permute}(Q, i_m)$

Looping over the modalities in query dimension

$y \leftarrow 0$

for $i \leftarrow 1$ to $|\phi_m|$ **do**

Intra-modality sparse attention computation for each modality w/ FlashAttention

$A_{mi} \leftarrow \text{softmax}(\text{sparse}(\bar{Q}_{mi} K^\top, i_{mi}) / \sqrt{d})$

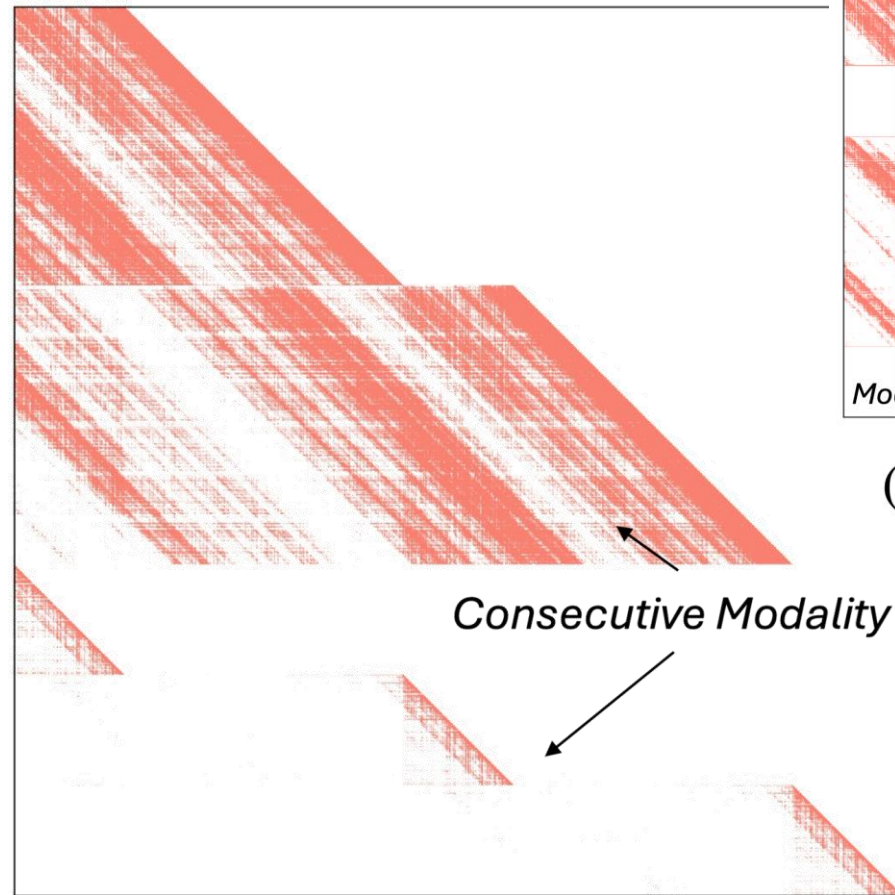
$y_{mi} \leftarrow \text{sparse}(A_{mi} V)$

Update the modality output to the final output

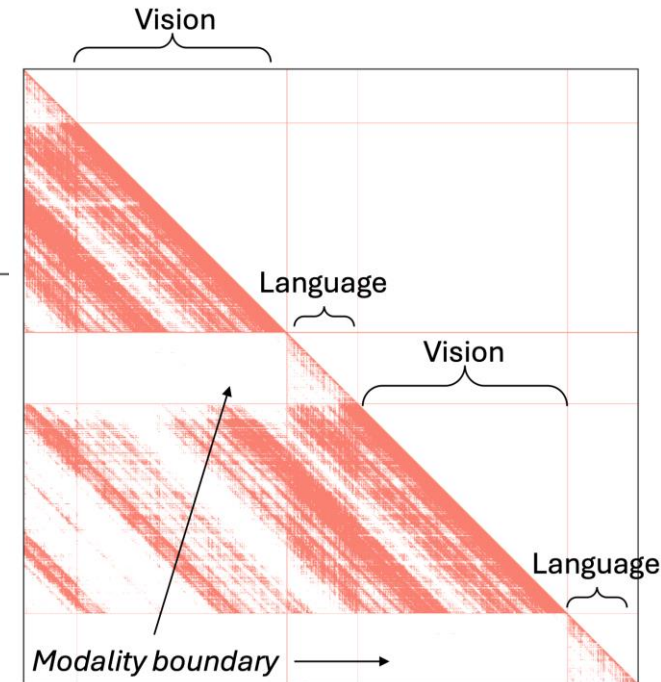
$y \leftarrow y_{mi} \cup y$

end for

return y



(e) Permuted Q-Boundary pattern.



(b) Q-Boundary pattern.

MMInference: 2D-Boundary pattern

Algorithm 3 2D-Boundary Head

Input: $Q, K, V \in \mathbb{R}^{S \times d_h}$, modality type index i_m , modality type set $m \in \phi_m$

Permute Q, K, V tensors based on modality

$\bar{Q} \leftarrow \text{permute}(Q, i_m), \bar{K} \leftarrow \text{permute}(K, i_m)$

$\bar{V} \leftarrow \text{permute}(V, i_m)$

Looping over the modalities in pairs

$y \leftarrow 0$

for $i \leftarrow 1$ to $|\phi_m|$ **do**

for $j \leftarrow 1$ to $|\phi_m|$ **do**

Dynamic sparse attention computation for each modality pair w/ FlashAttention

$m_{mi,mj} \leftarrow \text{buildmask}(i_{mi}, i_{mj})$

$A_{mi,mj} \leftarrow \text{softmax}(\text{sparse}(\bar{Q}_{mi} \bar{K}_{mj}^\top, i_{mi}, i_{mj}) / \sqrt{d} + m_{mi,mj})$

$y_{mi,mj} \leftarrow \text{sparse}(A_{mi,mj} \bar{V}_{mj})$

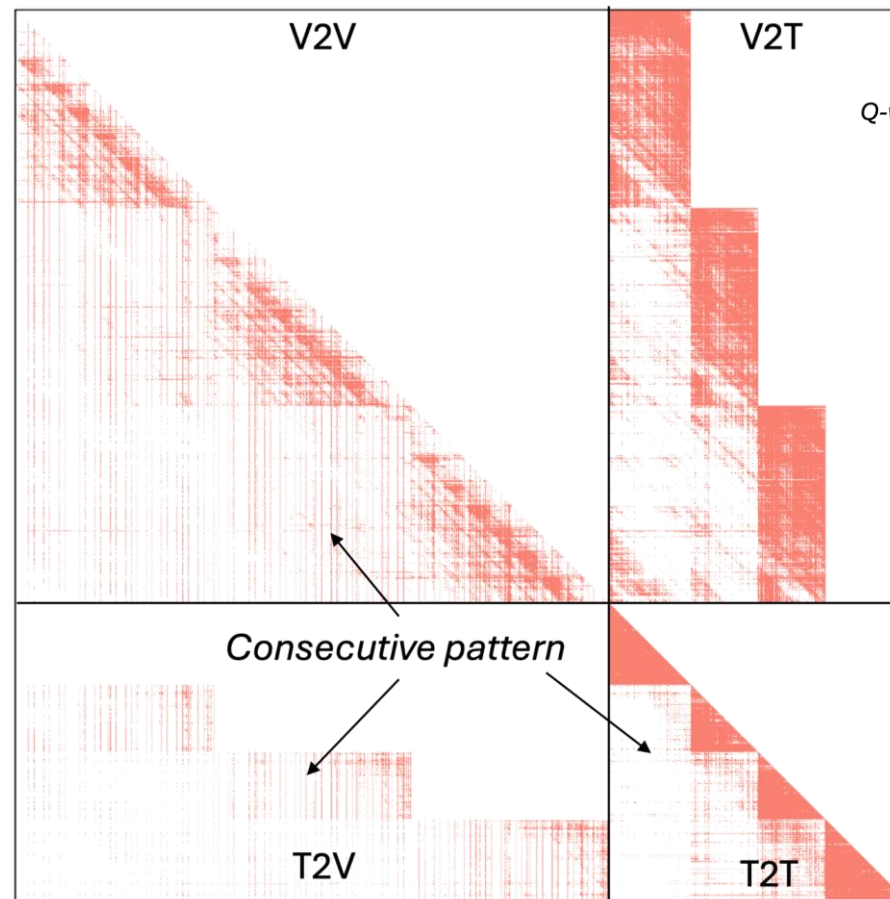
Update the modality output to the final output

$y \leftarrow y_{mi,mj} \cup y$

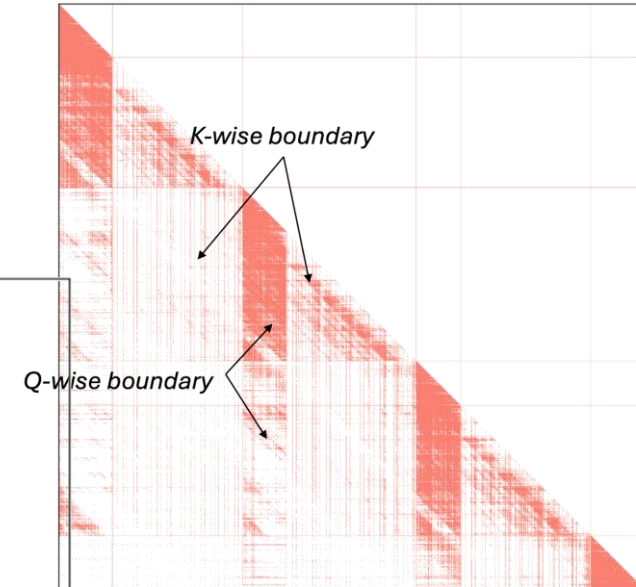
end for

end for

return y



(f) Permuted 2D-Boundary pattern.



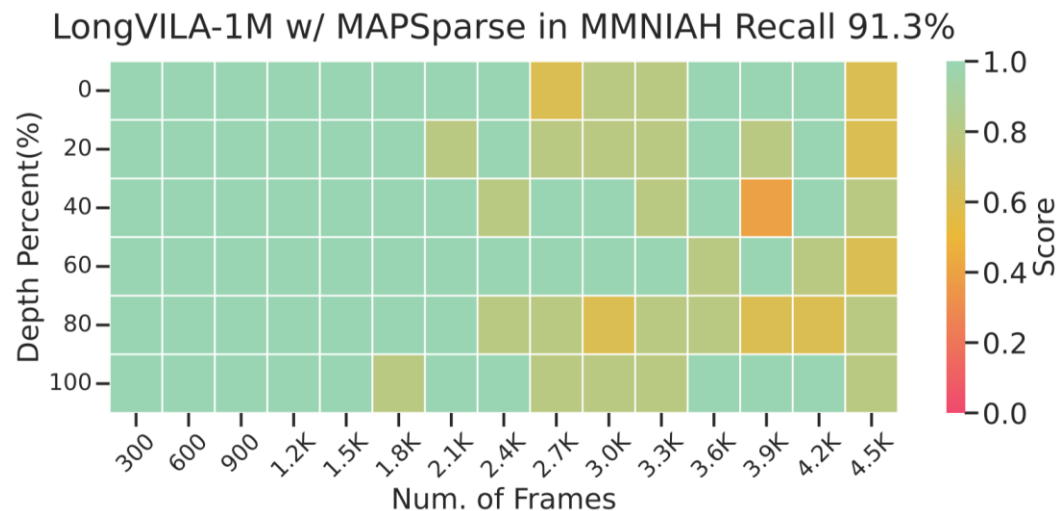
(c) 2D-Boundary pattern.

How effective is MMInference? Long-Video Benchmark

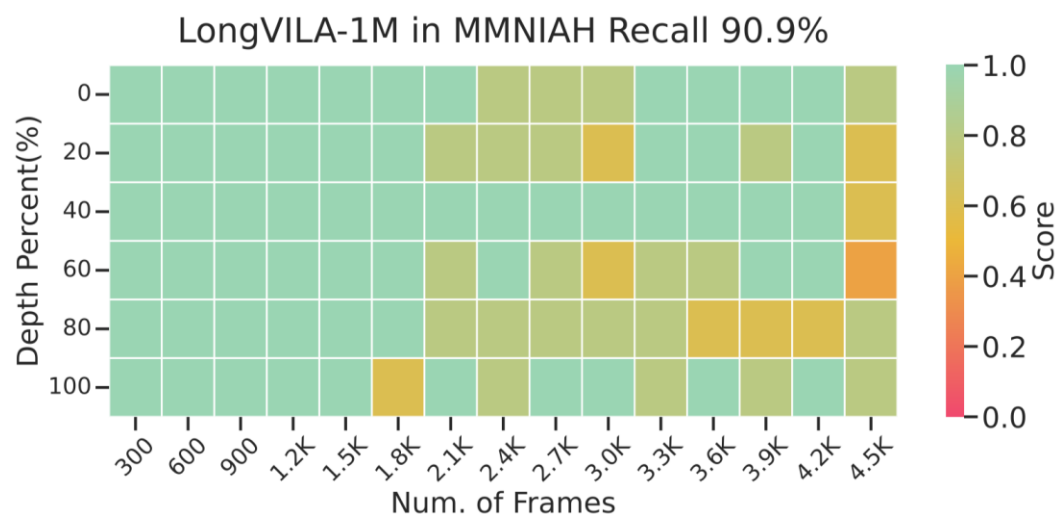
Table 1: Performance (%) of different models and different methods on video understanding tasks evaluated at frames from 110 to 256.

Model	FLOPs	VideoDC	ActNet-QA	EgoSchema	Next-QA	PerceptionTest	VideoMME		Avg.
		test	test	test	mc	val	wo/ sub.	w/ sub.	
Llava-Video-7B				# Frames: 110; Total # tokens: 20,240					
Full Attention	100%	3.66	59.6	57.0	81.2	66.1	64.7	71.0	57.6
SF-fixed	4.8%	3.26	57.3	53.3	79.8	62.9	59.9	67.1	54.8
SF-strided	41.4%	3.45	58.5	56.1	80.6	64.4	61.4	68.5	56.1
A-shape	48.2%	3.56	56.0	51.6	79.8	65.7	54.4	65.6	53.8
Tri-shape	49.0%	3.58	59.3	54.5	80.3	66.1	63.6	70.1	56.7
VisionZip	35.2%	1.35	42.1	40.5	69.5	41.4	44.9	62.1	43.1
MInference	78.8%	3.64	59.6	57.0	80.6	66.1	64.6	71.0	57.5
Ours	47.3%	3.58	59.8	57.1	80.1	66.2	64.5	71.8	57.6
LongVILA-7B				# Frames: 256; Total # tokens: 65,800					
Full Attention	100%	2.76	59.5	61.9	80.7	58.1	60.1	65.1	55.5
SF-fixed	2.2%	1.99	51.3	59.6	76.5	55.5	57.1	63.0	52.1
SF-strided	26.6%	2.58	56.0	61.4	76.7	55.5	53.6	59.2	52.2
A-shape	29.1%	2.75	56.6	60.9	75.0	55.3	49.1	59.6	51.3
Tri-shape	29.3%	2.63	58.1	62.0	77.8	56.2	59.3	63.3	54.2
VisionZip				OOM					
MInference	47.0%	2.77	59.7	62.2	79.1	57.8	60.0	65.2	55.2
Ours	31.8%	2.84	60.2	62.2	79.4	57.8	60.0	65.5	55.4

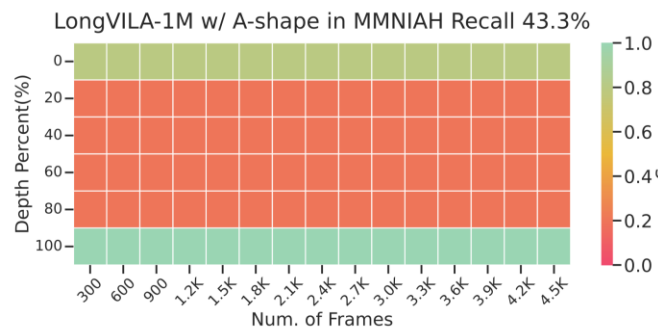
How effective is MMInference? V-NIAH



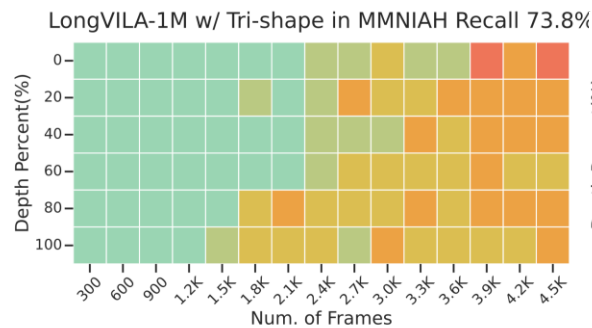
(c) MAPSparse in MM-NIAH



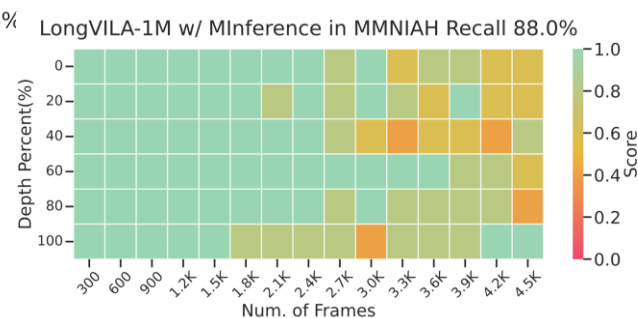
(d) FullAttention in MM-NIAH



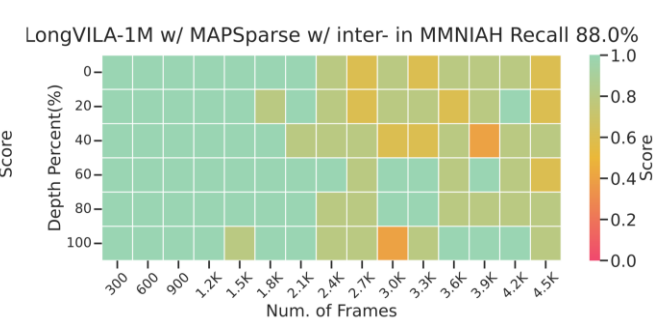
(a) A-shape



(b) Tri-shape

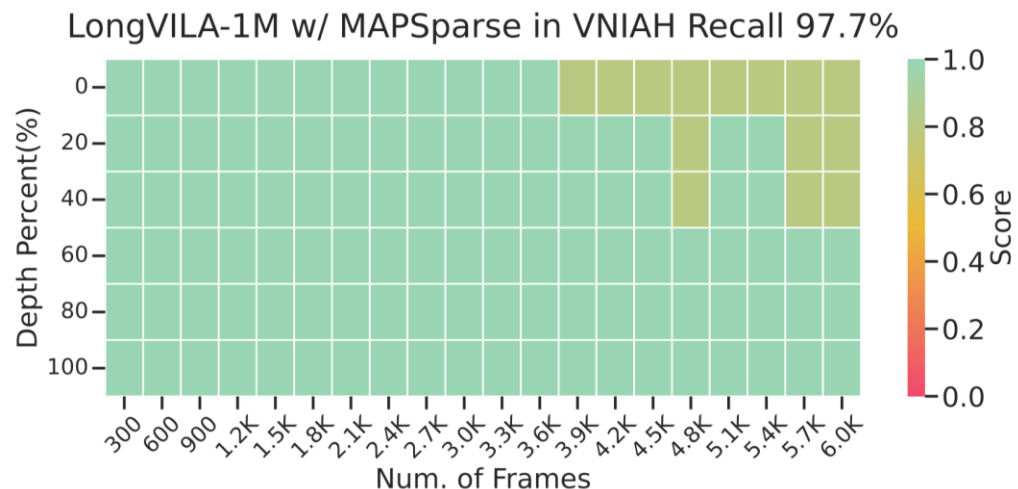


(c) MInference

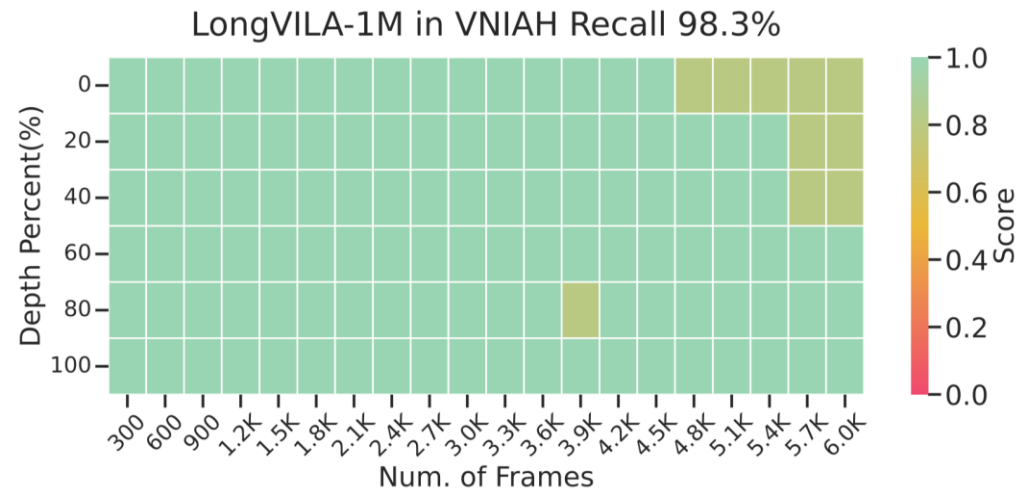


(d) MAPSparse w/ Inter-modality

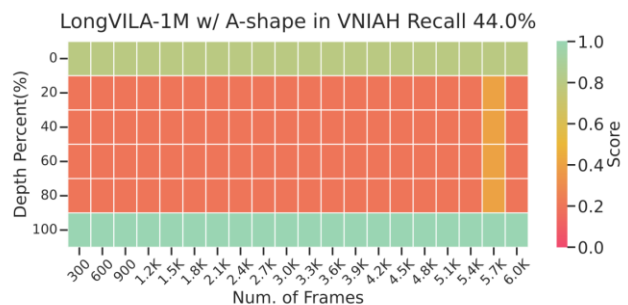
How effective is MMInference? MM-VIAH



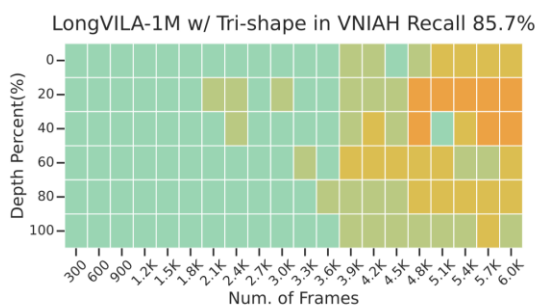
(a) MAPSparse in V-NIAH



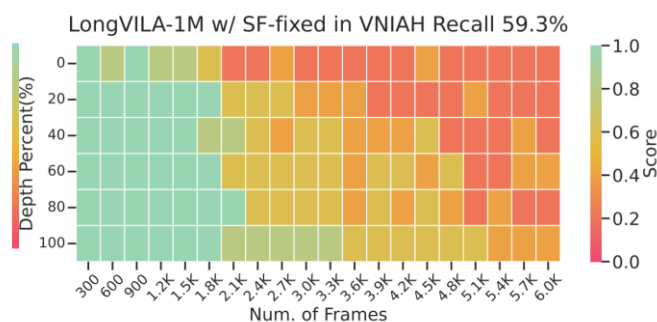
(b) FullAttention in V-NIAH



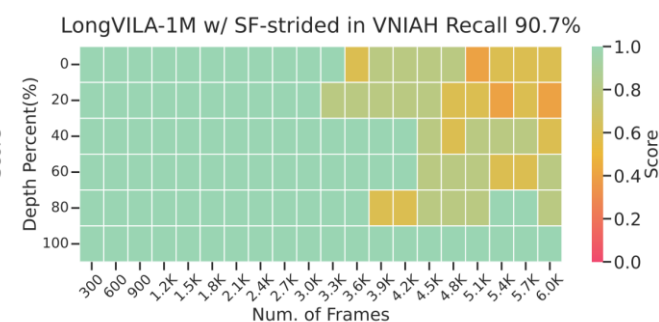
(a) A-shape



(b) Tri-shape

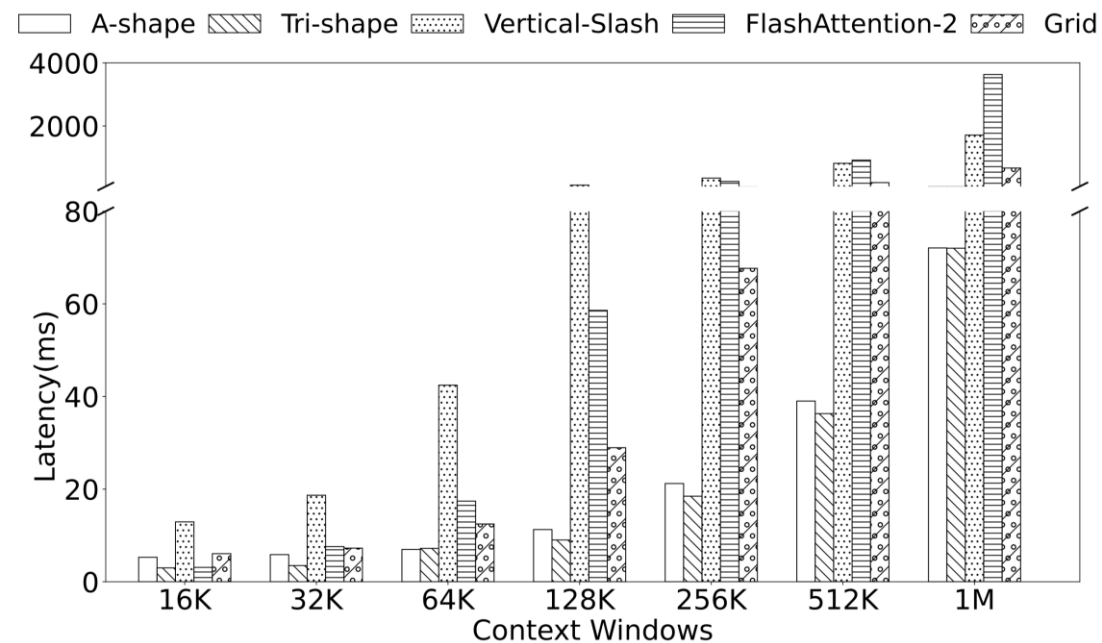
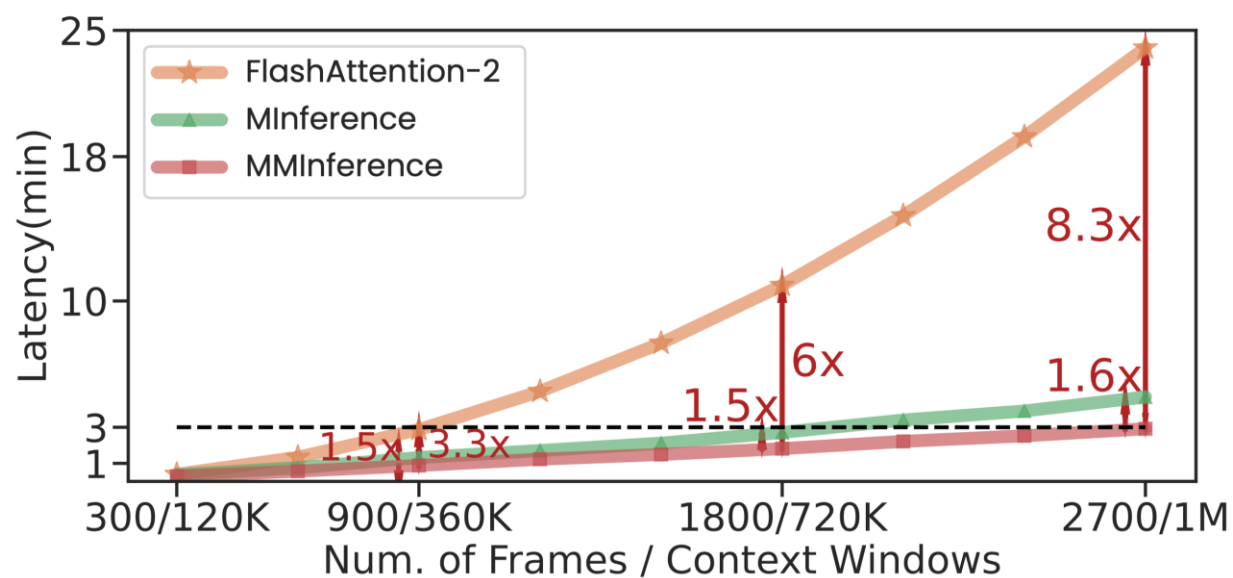


(c) SF-fixed



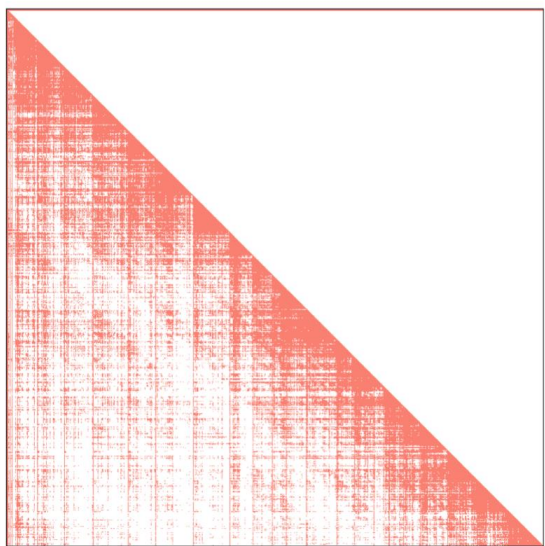
(d) SF-strided

How efficient is MMInference? – E2E & MicroBench

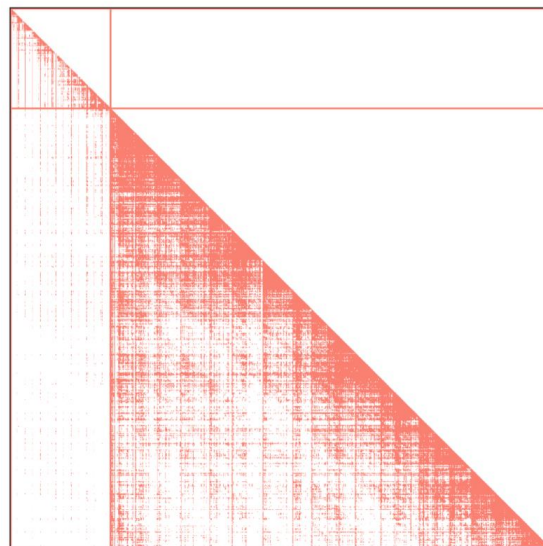


Transition of Sparse Patterns Across Modalities

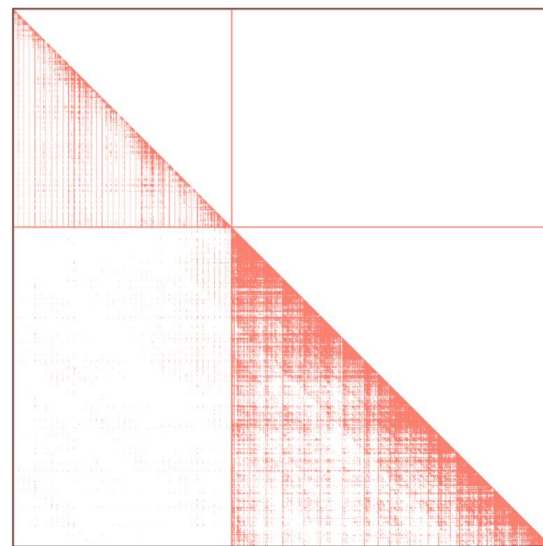
- ❑ The VS pattern shifts to a Grid pattern when the input transitions from text to visual.



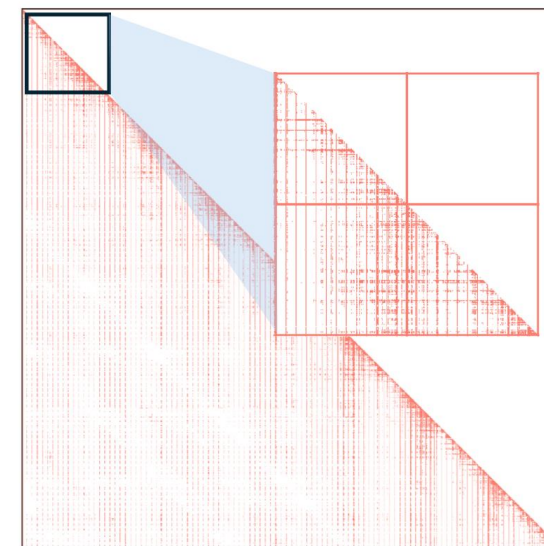
(a) All Textual Context



(b) Visual Context Inserted



(c) More Visual Context



(d) All Visual Context

Discussion

Algorithm 1 Sparse Attention

Input: $Q, K, V \in \mathbb{R}^{S \times d_h}$, $\tau \in [0, 1]$, $\gamma \in (0, 1)$

Determine the sparse pattern based on the threshold τ

$pattern \leftarrow \text{Sparse Pattern Search}(Q, K, \tau)$

Decide the sparse index set S based on pattern and γ

if $pattern == \text{query_specific}$ **then**

$S \leftarrow \text{Query Aware Index}(Q, K, \gamma)$

else if $pattern == \text{vertical_slash}$ **then**

$S \leftarrow \text{Vertical Slash Index}(Q, K, \gamma)$

end if

Compute the final sparse attention output

$y \leftarrow A(Q, K, V, S)$

return y

Algorithm 2 Sparse Pattern Search

Input: Q, K, τ

Take a representative query subset

$\hat{Q} = Q_{[-block_size:]}$

Compute estimated block-pooled attention $\bar{a}$ and true block-pooled attention $\hat{a}$

$\bar{a} \leftarrow \text{softmax}(\text{pool}(\hat{Q})\text{pool}(K)^T / \sqrt{d})$

$\hat{a} \leftarrow \text{pool}(\text{softmax}(\hat{Q}K^T / \sqrt{d}))$

Compute Jensen-Shannon divergence

$d_{JS} \leftarrow \sqrt{JSD(\bar{a} || \hat{a})}$

Determine whether to use Query-Specific attention pattern

if $d_{JS} < \tau$ **then**

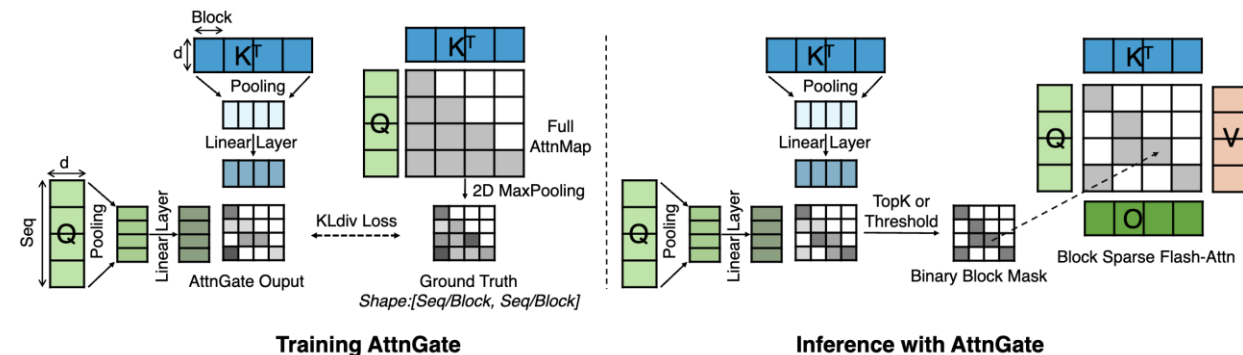
$pattern \leftarrow \text{query_specific}$

else

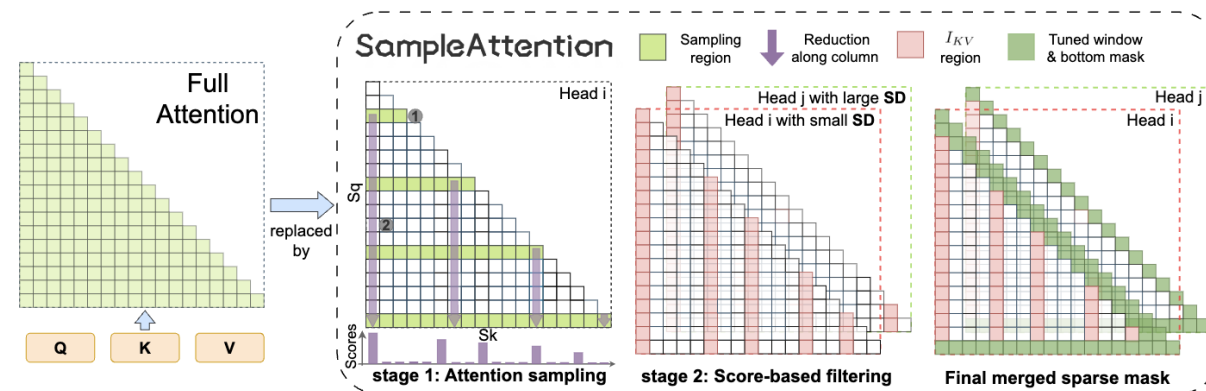
$pattern \leftarrow \text{vertical_slash}$

end if

return $pattern$



SeerAttention: Learned Gated



SampleAttention: topP + column

FlexPrefill: TopP

Discussion

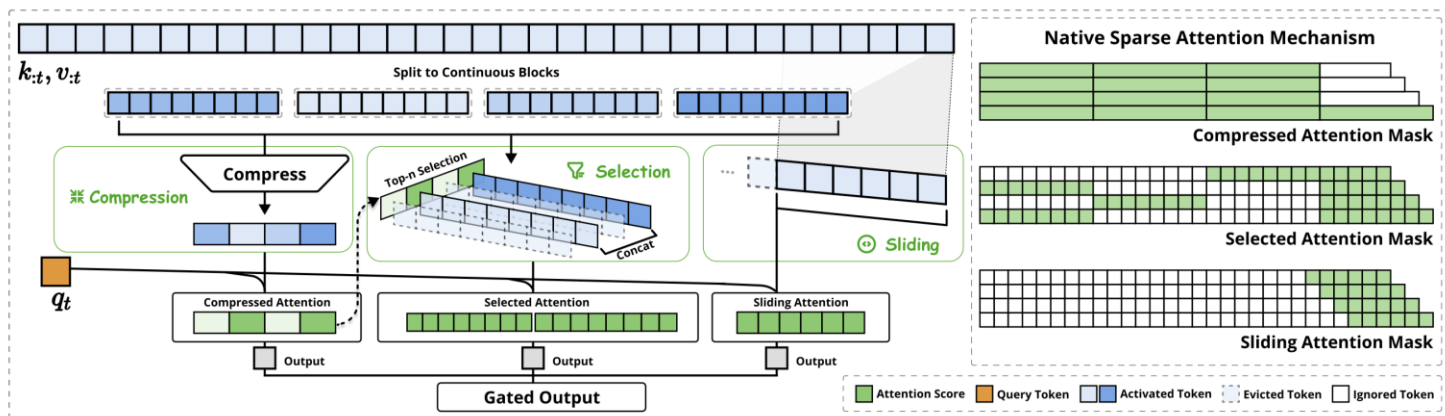
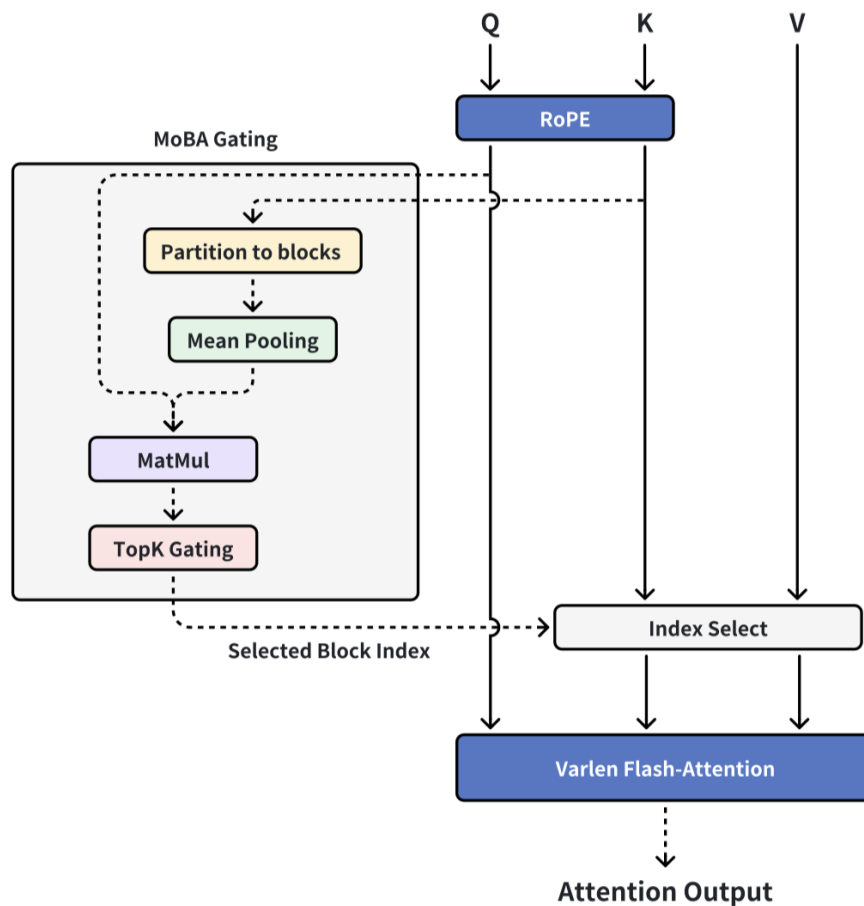
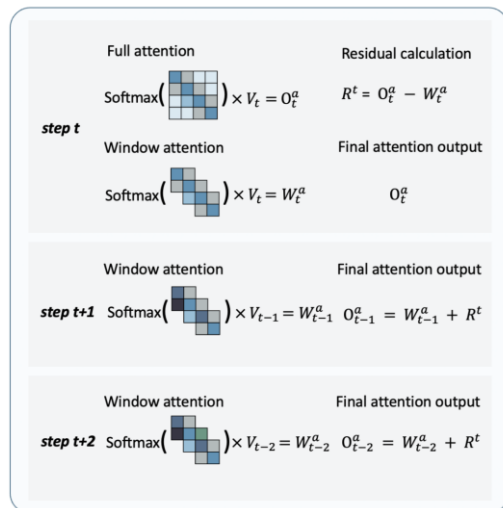


Figure 2 | Overview of NSA's architecture. Left: The framework processes input sequences through three parallel attention branches: For a given query, preceding keys and values are processed into compressed attention for coarse-grained patterns, selected attention for important token blocks, and sliding attention for local context. Right: Visualization of different attention patterns produced by each branch. Green areas indicate regions where attention scores need to be computed, while white areas represent regions that can be skipped.



Discussion



DiTFastAttn

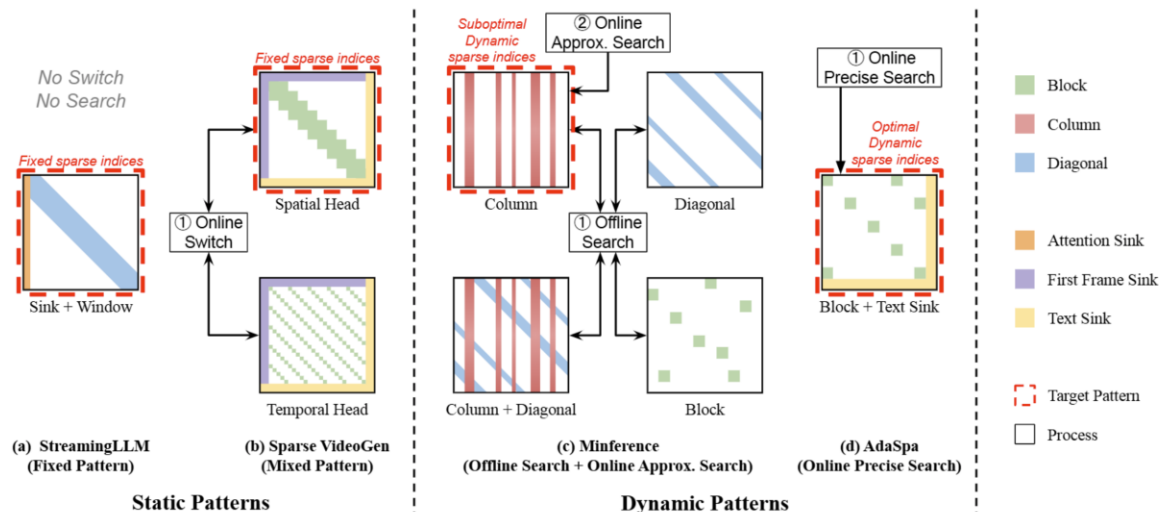


Figure 3. Different types of Sparse Pattern recognition methods. (a) StreamingLLM: using a static *sink+sliding window* pattern, need no search or switch. (b) Sparse VideoGen: preparing two predefined Static Patterns, and using an online switching method to determine which to use. (c) MInference: preparing several dynamic patterns, first do an offline search to determine the target pattern to use, then perform an online approximate search to search suboptimal sparse indices of this pattern. (d) AdaSpa: our method proves that the most suitable pattern for DiT is *blockified* pattern, and performs an online precise search to find the optimal sparse indices for blockified pattern.

Sparse VideoGen

AdaSpa

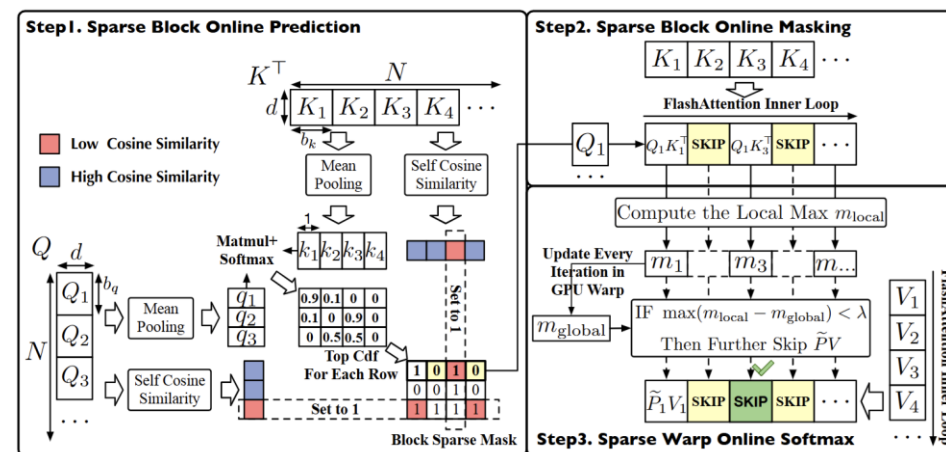
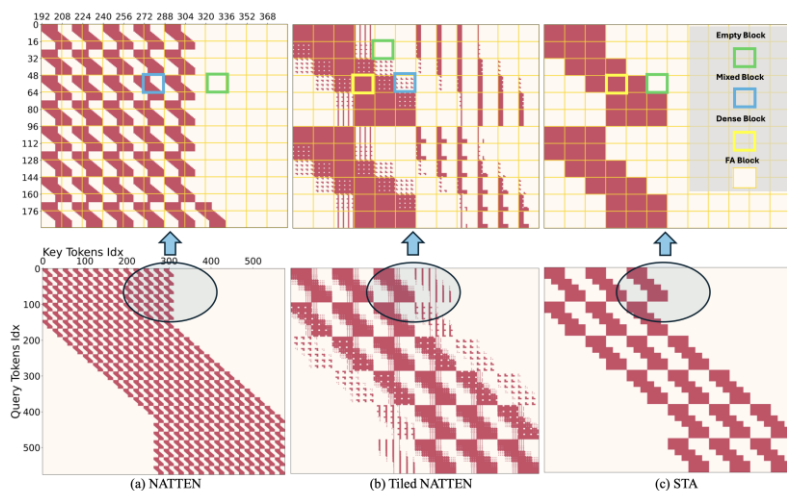


Figure 3. Workflow of SparseAttn.

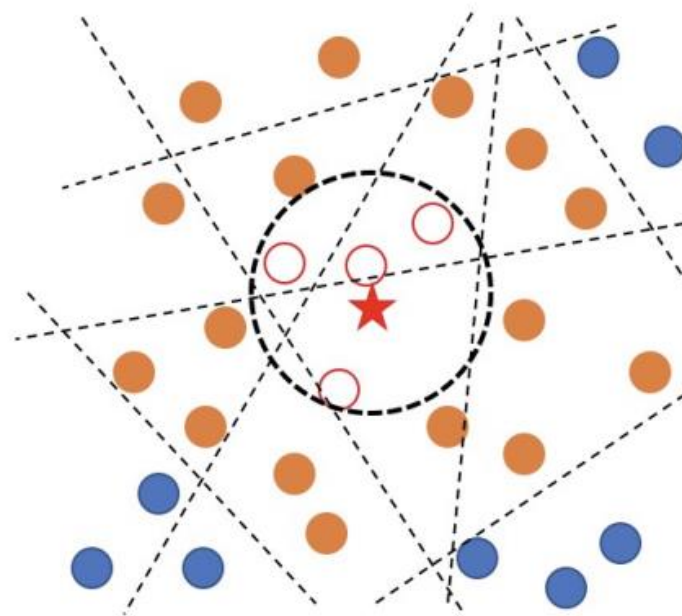
■ ANNS Is Semantically Aligned With Attention

- ANNS: Approximate Nearest Neighbor Search
 - Specifically, using inner-product as the distance metric, also known as Maximum Inner Product Search (MIPS).

$$\text{Attn}(q, K, V) = \text{Softmax}\left(\frac{qK}{\sqrt{d}}\right) V$$

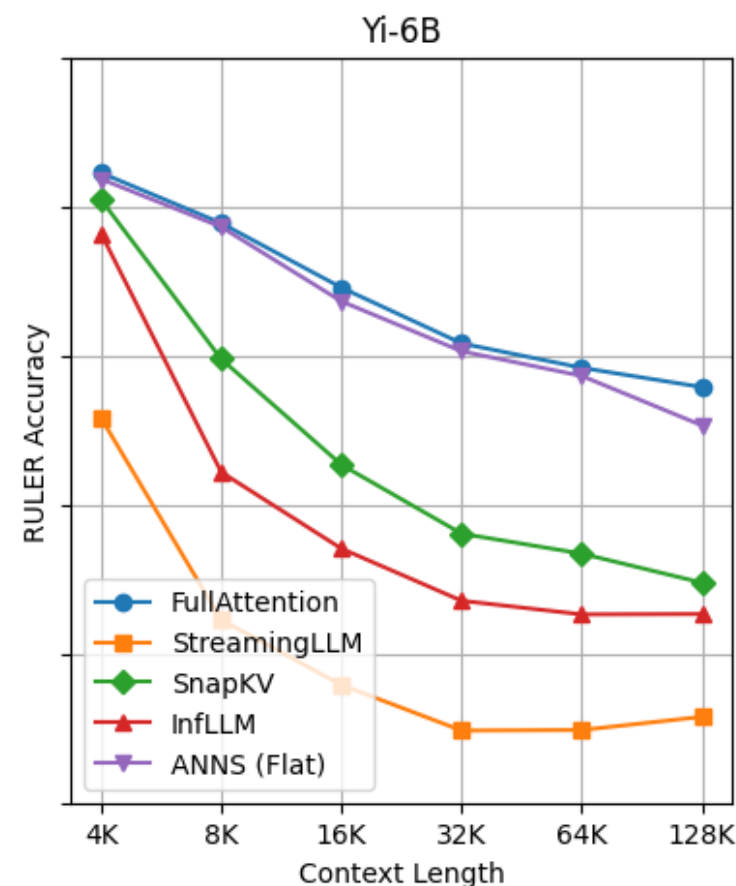
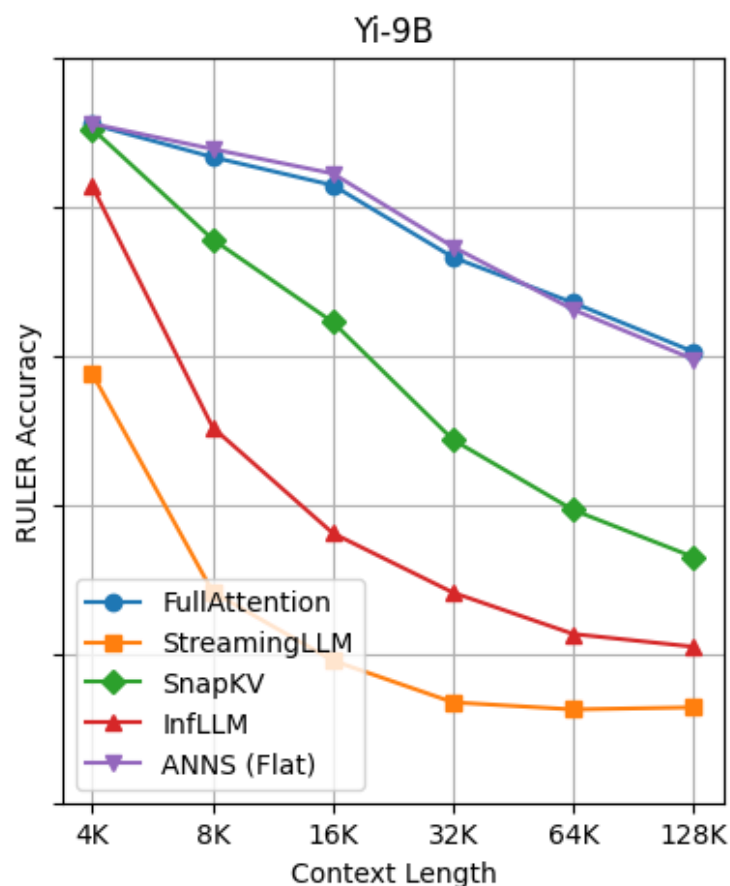
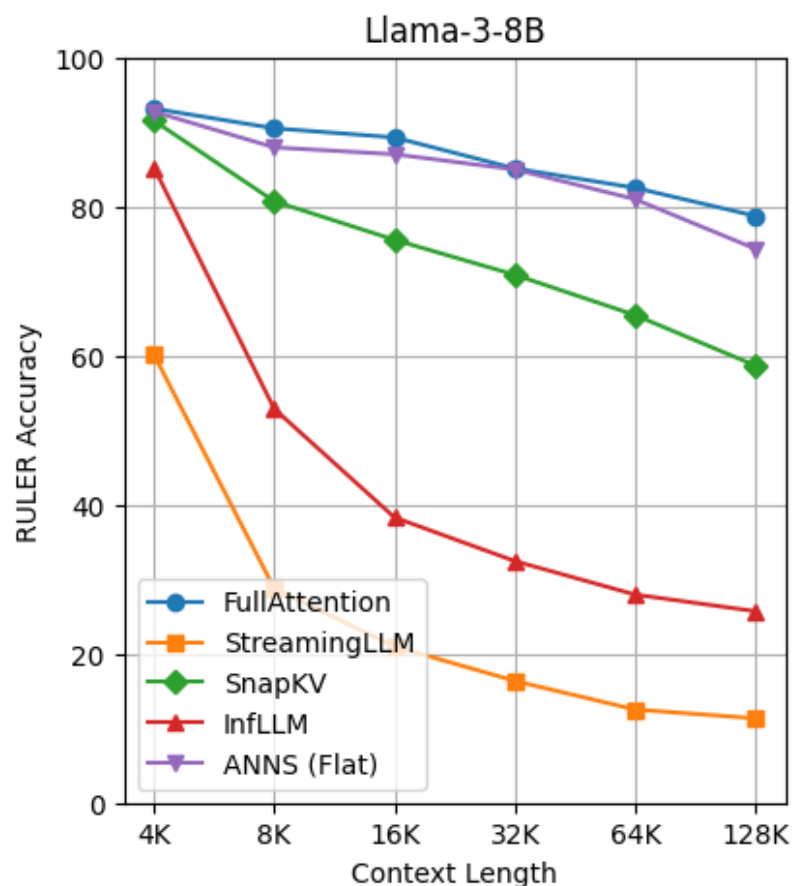
Inner-Product

Smooth one-hot argmax



Effectiveness of ANNS

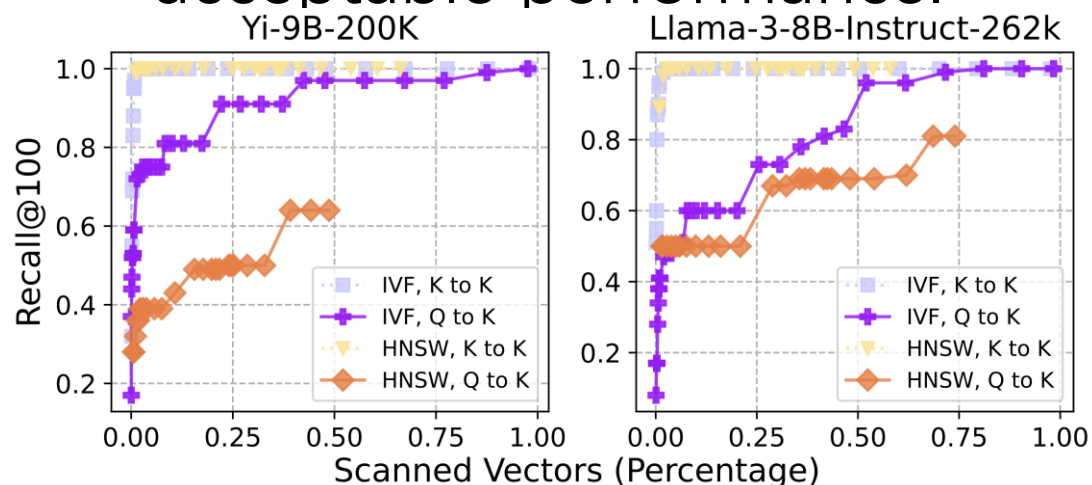
- Evaluation with RULER and ∞ -Bench



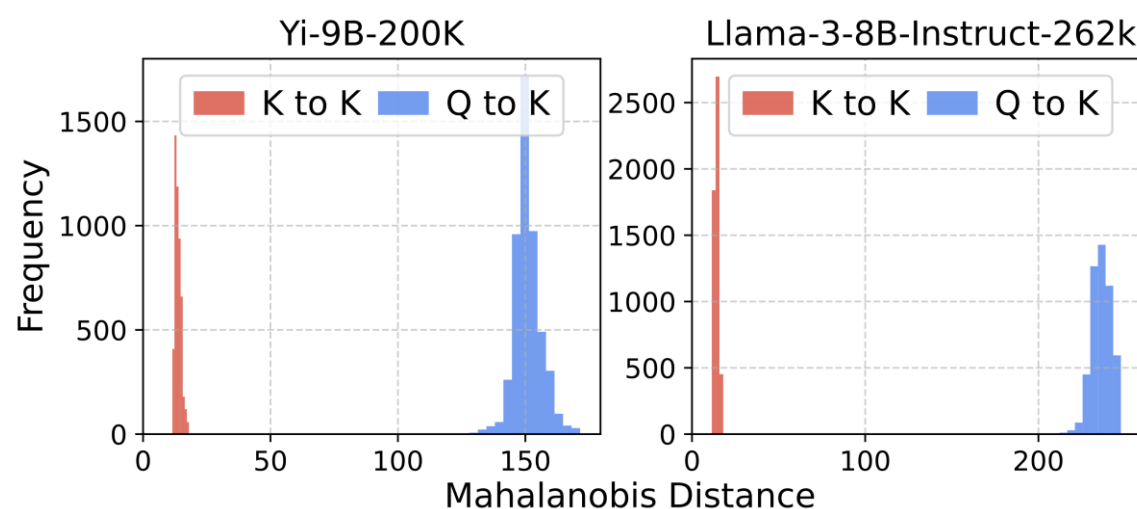
■ Observation 5: Out-of-Distribution Issues in Attention

□ Queries and keys have different distributions in attention. Off-the-shelf ANNS indexes perform poorly on $Q \rightarrow K$ searches while work well for $K \rightarrow K$ searches.

- about **30~50%** of key vectors are required to scan to maintain an acceptable performance.



(a) ANNS index performance.

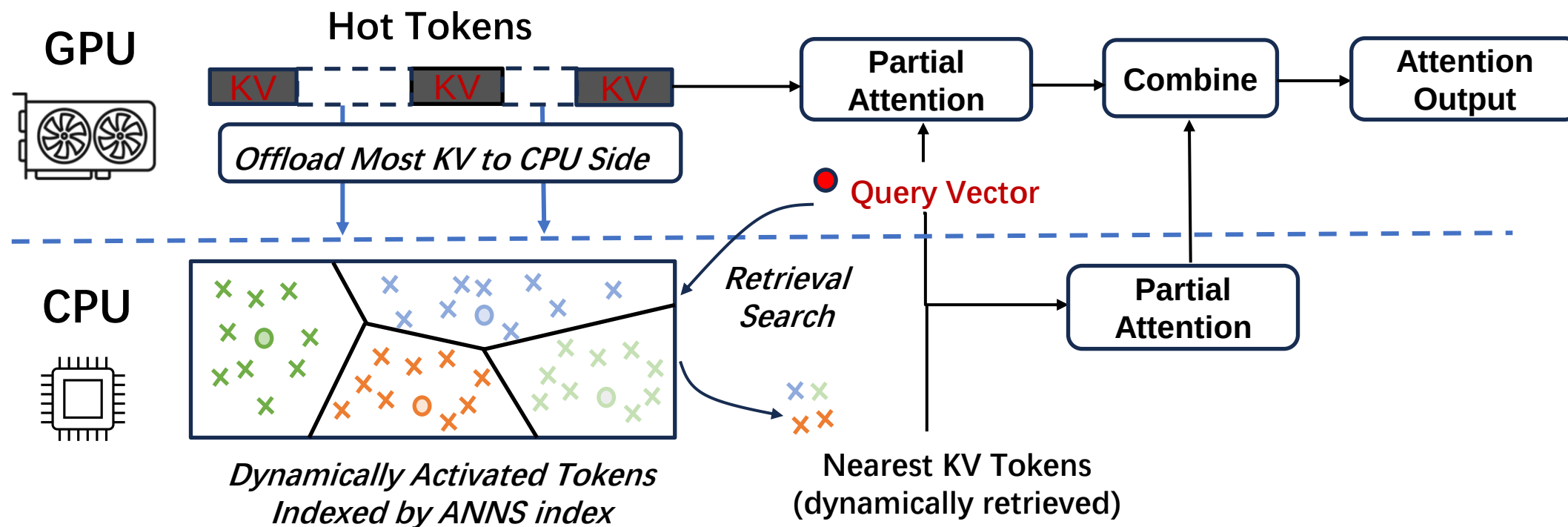


(b) Different distribution.

RetrievalAttention

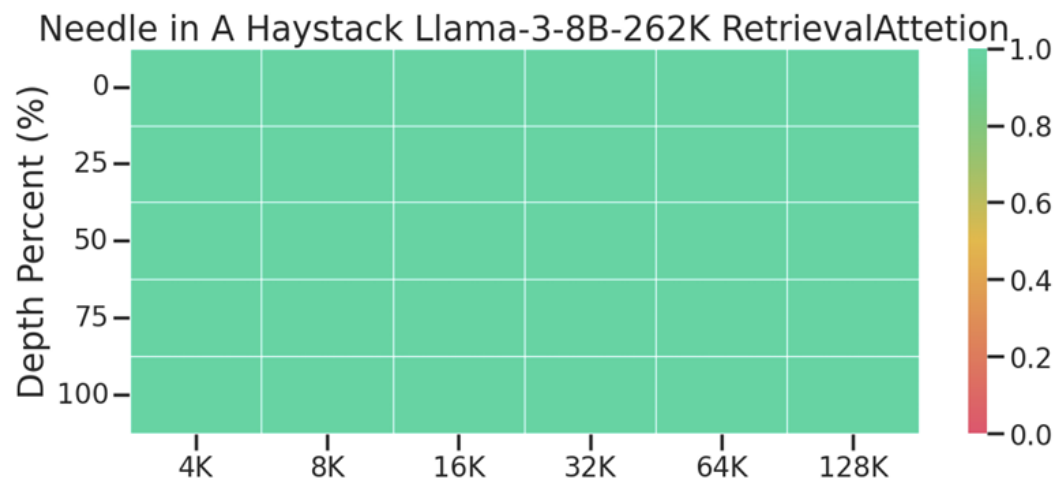
GPU requirement reduction by GPU-CPU Co-Execution: *Lower-End GPU + CPU = High-End GPU*

- ✓ Reduce memory access and data transfer with **OOD-aware** ANNS index
- ✓ Enable **128K** inference on **RTX 4090** with 5 tokens/second;



Highlighted Evaluation Results

- One NVIDIA RTX 4090 (24GB) to handle 128K tokens for LLMs with 8B parameters, generating a token in 0.188 seconds.



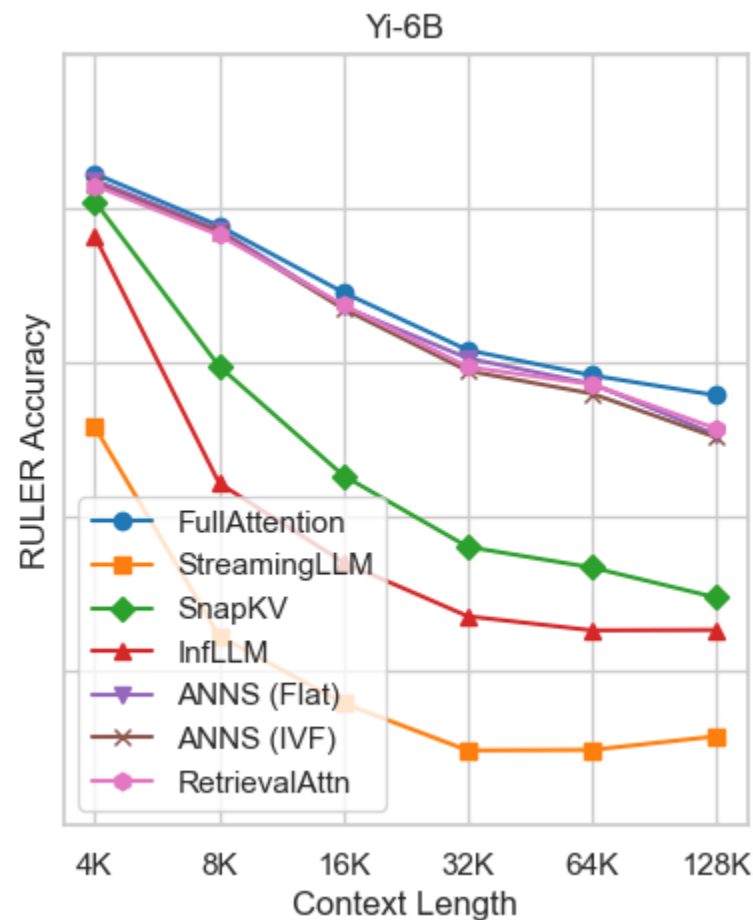
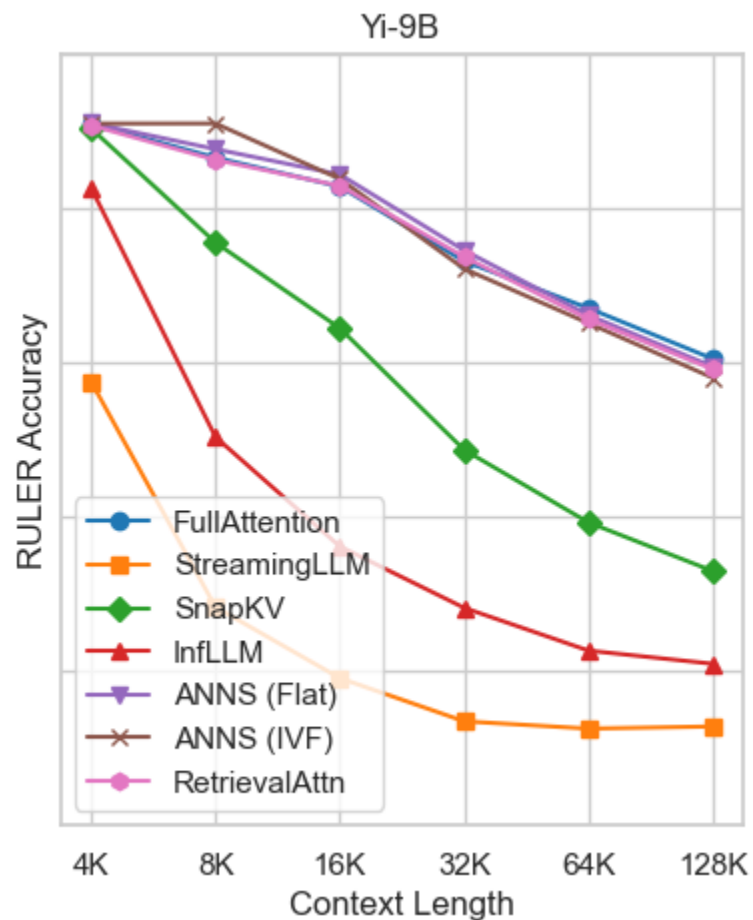
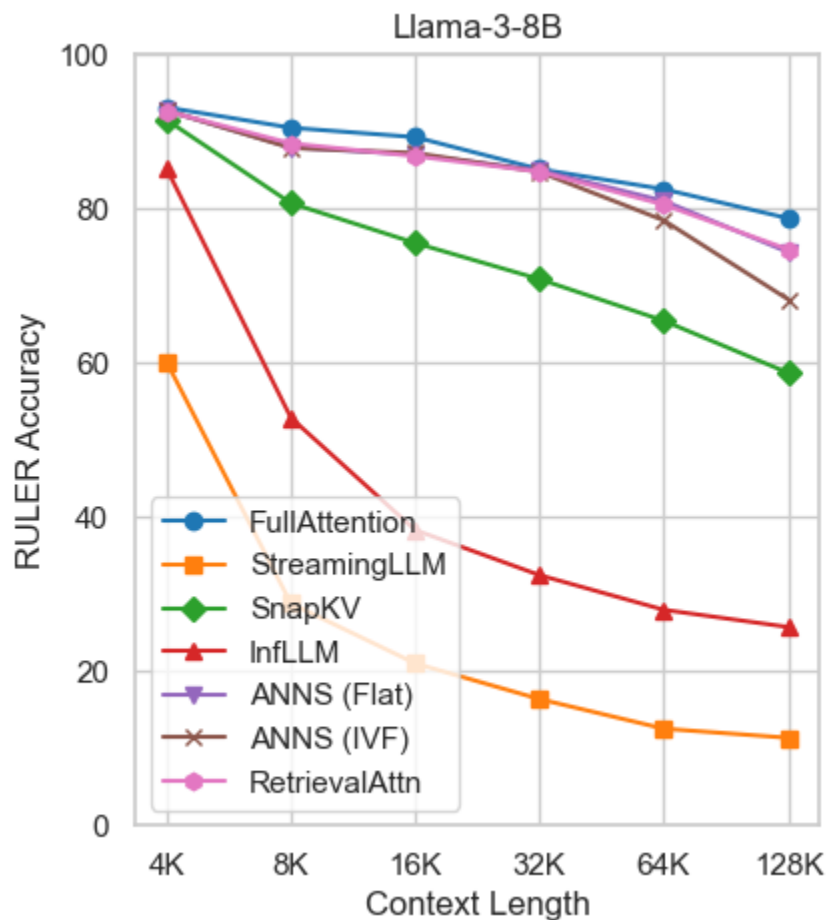
Methods	4K	8K	16K	32K	64K	128K
Full (without cache)	0.527	1.167	2.672	6.214	15.263	43.927
vLLM	OOM	OOM	OOM	OOM	OOM	OOM
StreamingLLM	0.029	0.030	0.029	0.030	0.030	0.029
SnapKV	0.029	0.028	0.028	0.029	0.029	0.028
InfLLM	0.058	0.063	0.063	0.065	0.067	0.069
MagicPIG	0.111	0.124	0.129	0.155	0.165	0.178
Flat	0.140	0.178	0.226	0.328	0.522	0.922
IVF	0.128	0.140	0.162	0.201	0.253	0.373
RetrievalAttention	0.132	0.139	0.151	0.157	0.164	0.183

Methods	100K	200K	500K	1M
Full (without cache)	25.47	83.03	457	1740
vLLM	0.029	OOM	OOM	OOM
StreamingLLM	0.034	0.035	0.032	0.035
SnapKV	0.035	0.035	0.034	0.034
InfLLM	0.082	0.079	0.082	0.084
MagicPIG	0.114	0.228	0.784	3.873
Flat	0.489	0.871	1.92	3.69
IVF	0.308	0.476	1.032	1.889
RetrievalAttention	0.155	0.167	0.170	0.172

40GB A100

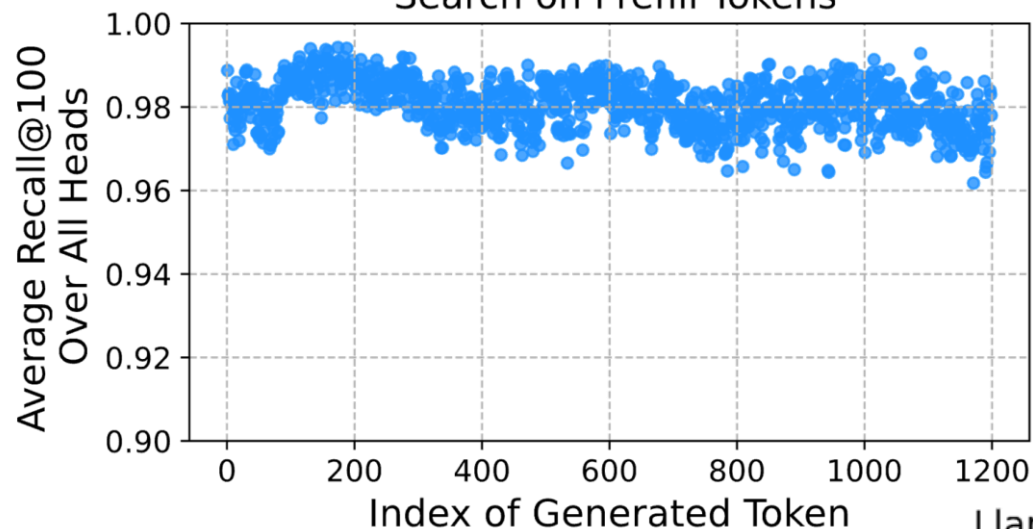
24GB RTX4090

Evaluation on RULER

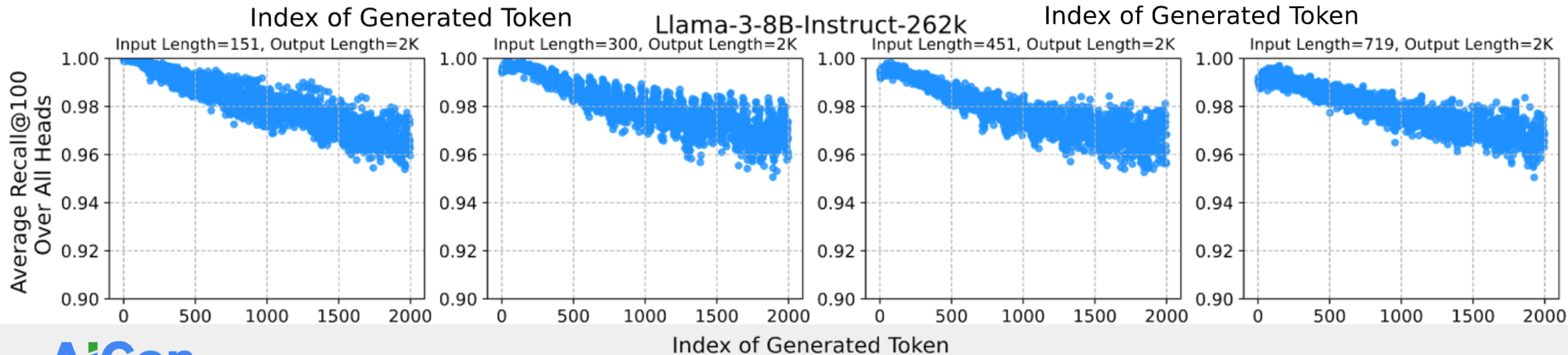
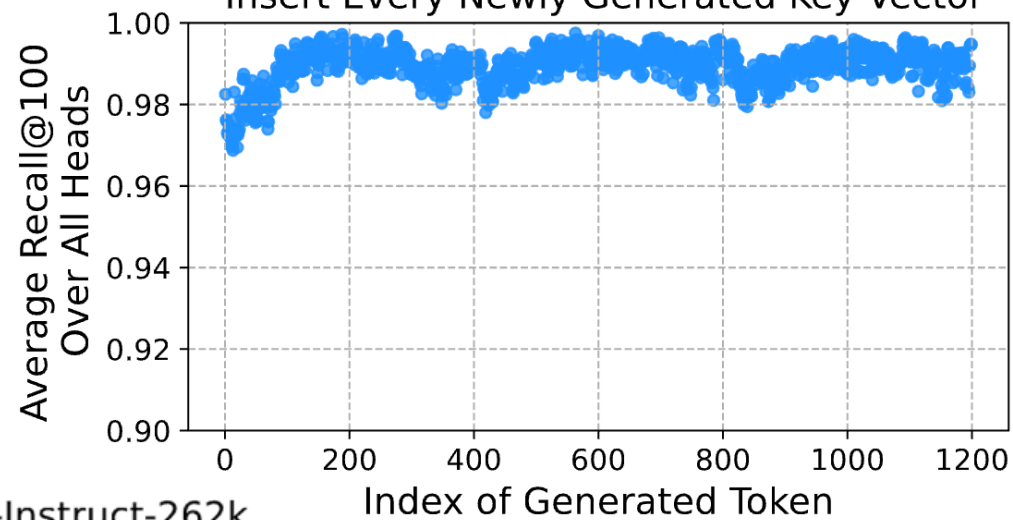


Long-generation

Llama-3-8B-Instruct-262k
Search on Prefill Tokens



Llama-3-8B-Instruct-262k
Insert Every Newly Generated Key Vector



05 总结与展望

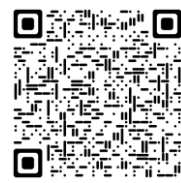


Microsoft

SCBench: A KV Cache-Centric Analysis of Long-Context Methods

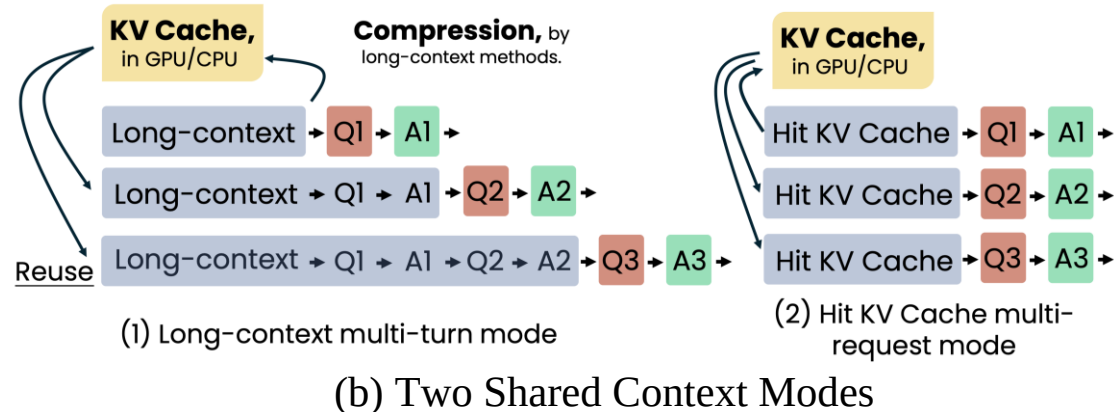
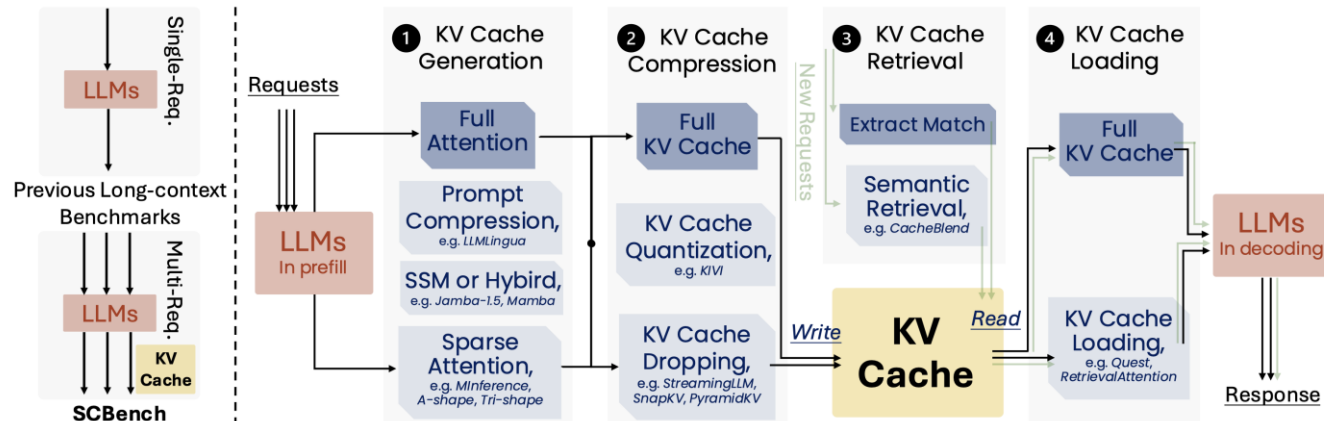


Project Page



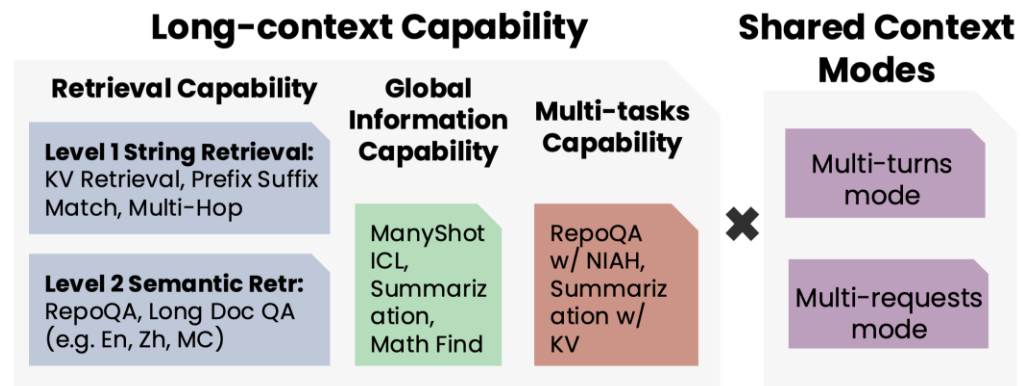
Code

Long-context methods are designed and utilized around the KV cache, but existing benchmarks focus only on single-request scenarios, ignoring its **full lifecycle in real-world use.**



We propose **SCBench**, a **KV cache-centric** benchmark for analyzing long-context methods, covering KV cache generation, compression, retrieval, and loading. It includes **four capability tasks** and **two shared-context modes**, from which we derive the following **insights**:

- *Sub- $O(n)$ memory is almost infeasible in multi-turn decoding;*
- *Task performance shows varying decline trends;*
- *All long-context methods experience performance degradation as the compression rate decreases;*
- *Long-generation scenarios exhibit distribution shift issues.*



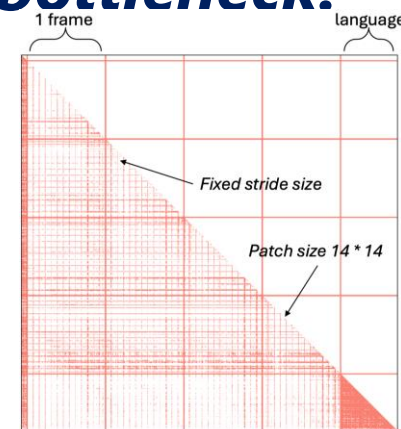
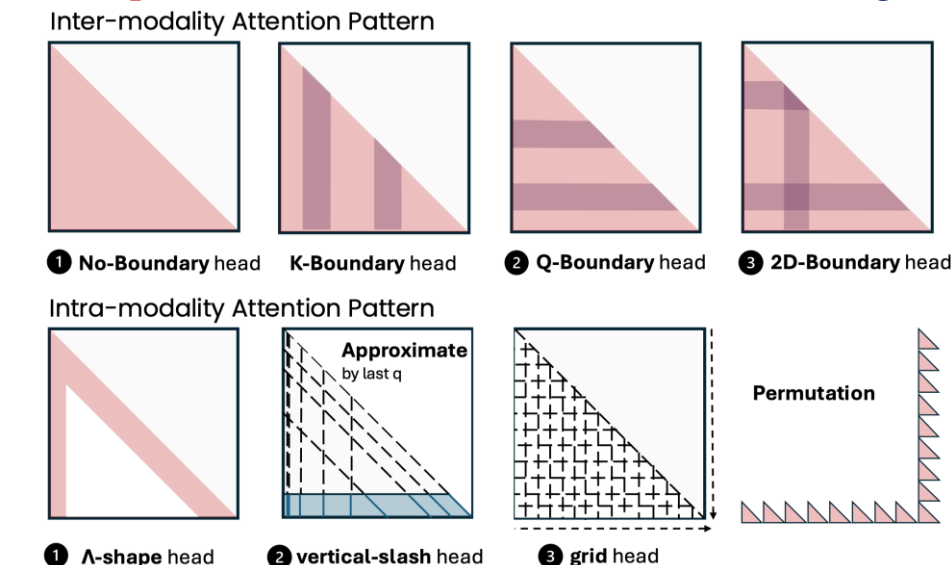
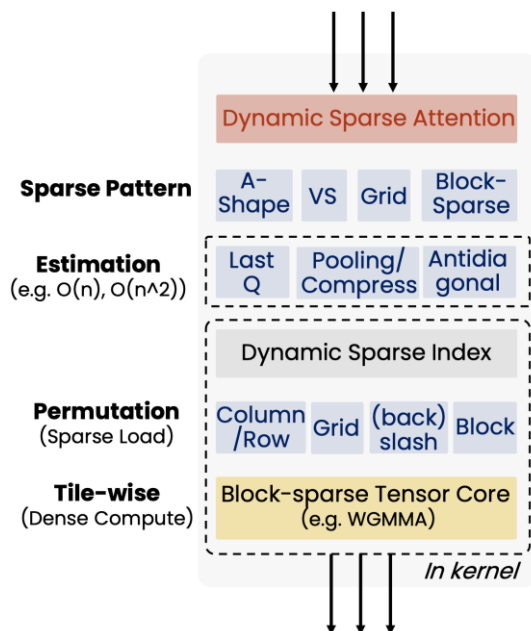
(c) Overview of SCBench



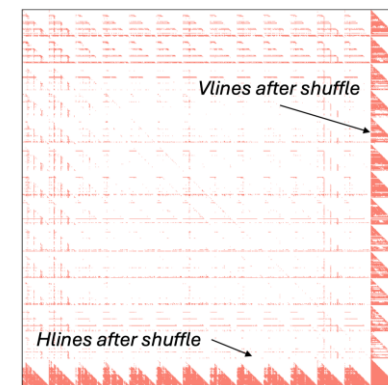
Microsoft

MMInference: Accelerating Pre-filling for Long-Context VLMs via Modality-Aware Permutation Sparse Attention

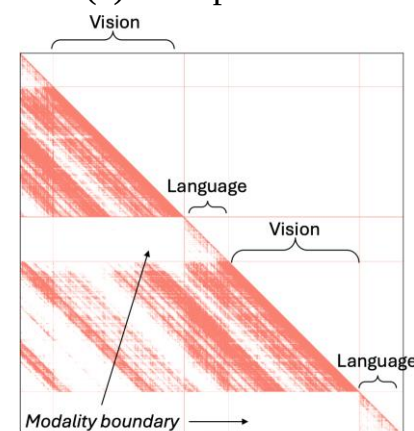
Long-context enables powerful *vision and multi-modal* applications, but *prefill cost* remains a major bottleneck.



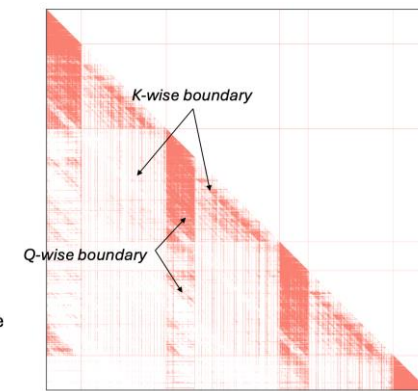
(a) Grid pattern.



(b) Permuted Grid pattern.



(c) Q-Boundary pattern.



(d) 2D-Boundary pattern.

We propose **MMInference**, a modality-aware permutation-based dynamic sparse attention method for multi-modal inputs. It introduces Grid-Shape and Q-/2D-Boundary sparse attention, achieving up to $8.3\times$ speedup without sacrificing performance. MMInference addresses two key challenges;

- *Vision-specific inductive bias* → handled via *Grid-Shape* attention;
- *Modality boundary in mixed inputs* → addressed by *Q-/2D-Boundary* attention;

RetroInfer: A Vector-Storage Approach for Scalable Long-Context LLM Inference

We build KV Cache as a **Vector Storage System** in a CPU-GPU co-execution setup (Fig1) to accelerate long-context LLM inference **without model accuracy loss** (Fig3). The core of RetroInfer is as follows:

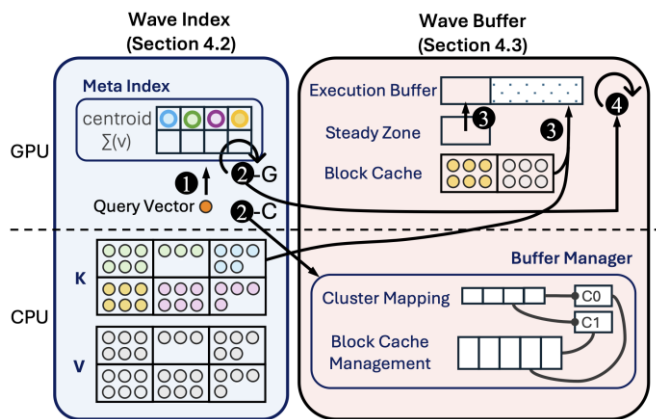


Fig1. Architecture of RetroInfer

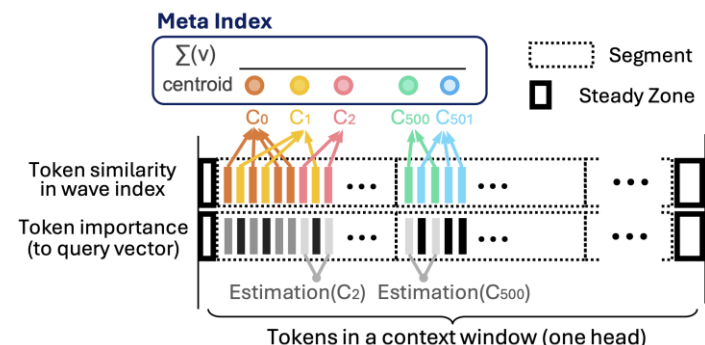


Fig2. Attention-aware Design of Wave Index

- An **Attention-aware V**ector index named **wave Index** (Fig2), which adopts tripartite attention approximation and accuracy-bound attention estimation to fit the dynamic sparsity of attention
- A **wave buffer**, which coordinates KV cache placement and overlaps computation and data transfer across GPU and CPU to sustain high throughput.

Powered by the wave index and wave buffer, the decoding throughput of RetroInfer outperforms baselines by **4.5x-10.5x** across different context lengths while it is the **only solution** that matches full attention accuracy.

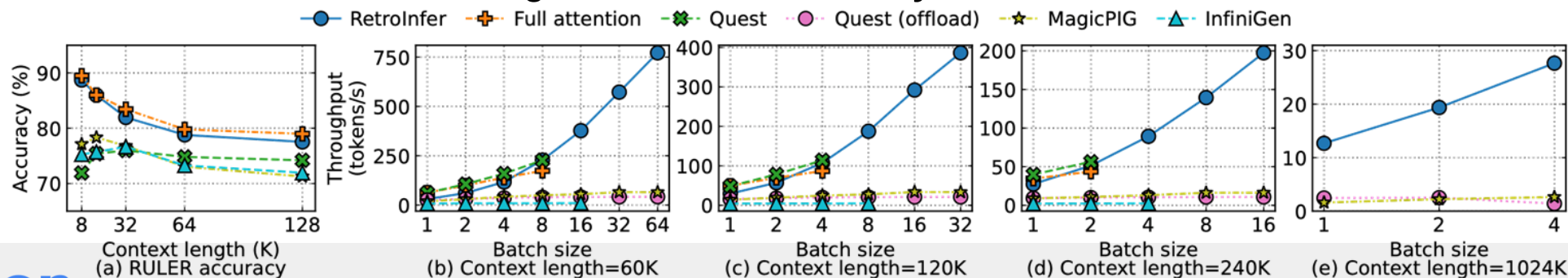


Fig3. Model Accuracy and Decoding Throughput Comparison

Achievement

-  • **Achieves 10x speedup for 1M tokens inference TTFT** while maintaining full attention performance;
-  • **MI**nference accept by NeurIPS'24 as **Spotlight**, **SCBench** and **MMInference** accept by **ICLR'25** and **ICML'25**;
-  • RetrievalAttention awarded as Best Paper Award in ENLSP-IV @ NeurIPS'24;
-  • Adopted by **vLLM** and **SGLang**, two widely used business-level LLM inference framework;
-  • Used by official **services** of open-sourced long context models, such as **Qwen-Turbo-1M** and **Seed**.

<https://github.com/sgl-project/sglang/pull/5327>
<https://github.com/vllm-project/flash-attention/pull/33>

Qwen2.5-1M Technical Report
FlexPrefill: A Context-Aware Sparse Attention Mechanism for Efficient Long-Sequence Inference

Future Direct

- **Efficient Long Context Extension Method;**
- **Efficient Long Generation Method;**
- **Efficient RL Training;**
- **Native Efficient Architecture;**

📍 北京

QCon

全球软件开发大会

会议时间：4月10-12日

- 大模型赋能 AIOps
- 越挫越勇的大前端
- 云上业务架构演进
- 多模态大模型及应用
- 海外 AI 应用创新实践

📍 北京

AiCon

全球人工智能开发与应用大会

会议时间：6月27-28日

- 端侧智能
- AI Agent
- 多模态大模型
- 金融+大模型

📍 上海

QCon

全球软件开发大会

会议时间：10月23-25日

- AI Agent
- 大模型训练推理
- 端侧 AI
- 搜推深度融合
- 数智金融

4月

5月

6月

8月

10月

12月

📍 上海

AiCon

全球人工智能开发与应用大会

会议时间：5月23-24日

- 通用大模型
- AI Agent
- 垂直领域应用
- 模型可解释性

📍 深圳

AiCon

全球人工智能开发与应用大会

会议时间：8月22-23日

- 模型效率与部署
- 多模态大模型
- 大模型安全
- 智能硬件

📍 北京

AiCon

全球人工智能开发与应用大会

会议时间：12月19-20日

- 通用大模型
- 智能硬件
- LMOPs
- 具身智能

极客邦科技 2025 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询



查看会议

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会